U N I V E R S I D A D S I G L O 2 1

D e m o c r á t i c a · L a i c a · T r a s c e n d e n t e

Seminario de Practica de Informática

Trabajo Práctico N° 4

Alumno: *Beber Feil Thiago*

Fecha de Realización: *3/11/2026*

Titular Experto: *Marcos Dario Aranda*

Titular Disciplinar: *Pablo Virgolini*

Sistema de Gestión de Ventas y Garantías para PuntoTecno

TP 1

Introducción

PuntoTecno es un local dedicado a la comercialización de electrodomésticos, que atiende diariamente a un volumen considerable de clientes que adquieren productos de distintas categorías (línea blanca, pequeños electrodomésticos, tecnología del hogar, entre otros). Cada producto vendido cuenta con un período de garantía que varía según el tipo de artículo y el fabricante, lo cual exige un control preciso de las fechas de venta y de vencimiento de cada garantía.

Actualmente, el registro de ventas y el seguimiento de las garantías se realiza de manera manual o mediante planillas no integradas, lo que dificulta contar con información centralizada, actualizada y confiable al momento de atender consultas de clientes o de gestionar la trazabilidad de un producto vendido.

En este contexto, se plantea el desarrollo de un sistema informático que permita digitalizar y centralizar el proceso de registro de ventas y de gestión de garantías, brindando a la organización una herramienta ágil para conocer en todo momento el estado de cada producto vendido y facilitar la atención al cliente.

Adicionalmente, el desarrollo de este proyecto se enmarca en la aplicación del Proceso Unificado de Desarrollo (PUD) como metodología de trabajo, empleando Java como lenguaje de programación y MySQL como motor de base de datos relacional para la persistencia de la información, conforme a los lineamientos establecidos para el presente trabajo práctico.

Justificación

PuntoTecno cuenta con una trayectoria consolidada en la venta de electrodomésticos y busca fortalecer su posicionamiento en un mercado cada vez más competitivo, donde la calidad de atención al cliente y la eficiencia en la gestión posventa constituyen factores diferenciales frente a otros comercios del rubro.

La gestión de garantías es un aspecto crítico dentro de la operatoria del negocio, ya que de ella depende en gran medida la confianza que el cliente deposita en el local al momento de realizar una compra. Un correcto seguimiento de este proceso permite a la empresa responder con precisión ante consultas, evitar errores en la determinación de la vigencia de una garantía y sostener una imagen de seriedad y profesionalismo.

La gestión de la relación con el cliente resulta clave para la sostenibilidad de cualquier negocio comercial, ya que un cliente que percibe seguridad y respaldo en su compra tiende a generar mayor fidelización y recomendación del establecimiento.

El proyecto para implementar un sistema de gestión de ventas y garantías en PuntoTecno requerirá la incorporación de herramientas tecnológicas para el registro y procesamiento de la información, pero también contribuirá a ordenar y optimizar un proceso que hoy se realiza de forma manual, reduciendo el margen de error humano y el tiempo destinado a tareas administrativas.

La relevancia del proyecto se extiende a nivel operativo y comercial. A medida que el volumen de ventas del local crece, la falta de un sistema centralizado se traduce en mayores dificultades para localizar información histórica de ventas y garantías, lo que impacta directamente en la calidad de atención brindada. Con la implementación del sistema propuesto, se espera optimizar la gestión interna del local, mejorar los tiempos de respuesta ante consultas de clientes.

Definiciones del proyecto

Objetivo general del proyecto

Desarrollar un sistema de gestión de ventas y garantías que permita centralizar y automatizar el registro de las operaciones comerciales de PuntoTecno, optimizando el control de vigencia de garantías y mejorando la calidad de atención al cliente.

Objetivos específicos

1. Diseñar e implementar un módulo de registro de ventas que permita asociar cada operación con el producto, el cliente y el vendedor correspondiente, reduciendo los errores derivados de la carga manual de información.
2. Automatizar el cálculo y la asignación del período de garantía de cada producto vendido, en función de su categoría, para eliminar el cálculo manual y disminuir el margen de error en la determinación de su vigencia.

3. Facilitar la consulta centralizada del historial de ventas y del estado de garantía de los productos, tanto por cliente como por artículo, para agilizar los tiempos de respuesta ante consultas.

Definiciones del sistema

Objetivo general del sistema

Registrar y gestionar de forma centralizada las ventas realizadas por PuntoTecno y las garantías asociadas a cada producto vendido, permitiendo consultar en todo momento su estado de vigencia.

Límites, alcances y restricciones del sistema

Alcances:

- El sistema permitirá registrar ventas indicando producto, cliente, vendedor y fecha de la operación.
- Calculará y asignará automáticamente la garantía correspondiente a cada venta, según la categoría del producto.
- Permitirá consultar el estado de vigencia (vigente/vencida) de la garantía de un producto determinado.
- Permitirá consultar el historial de ventas y garantías por cliente o por producto.

Límites:

- El sistema no contemplará, en esta etapa, la gestión de reclamos ni de service técnico (según lo indicado por la cátedra).
- No incluirá gestión de inventario ni de stock de productos.
- No gestionará pagos ni facturación electrónica; se asume que estos procesos se realizan por fuera del sistema.

Restricciones:

- El sistema debe desarrollarse utilizando Java como lenguaje de programación.
- Debe emplear MySQL como motor de base de datos relacional para la persistencia.
- Debe seguir los lineamientos del Proceso Unificado de Desarrollo (PUD) a lo largo de todas las entregas.
- Para esta primera entrega, la implementación se limitará a un prototipo operacional, sin persistencia real en base de datos (esto se incorporará en los trabajos prácticos posteriores).

Elicitación

Para este proyecto, al tratarse de un sistema pensado para un negocio en el que trabajo actualmente, se definió combinar dos técnicas complementarias: la **observación directa** del proceso de venta y la **entrevista** a los actores involucrados en la operatoria diaria del local.

- **Observación directa:** al desempeñarme en PuntoTecno, tuve la posibilidad de participar y presenciar de forma cotidiana el proceso de venta y la forma en que actualmente se registra la información de garantías, lo cual permitió identificar de primera mano las falencias del proceso manual y las necesidades reales del negocio, sin depender exclusivamente del relato de terceros.
- **Entrevistas:** se realizaron entrevistas breves y semiestructuradas al encargado del local y a otro vendedor, con el objetivo de profundizar en aspectos que no surgen de la sola observación, como los criterios de asignación de garantía según el tipo de producto, los problemas más frecuentes que se presentan al momento de informar a un cliente sobre el estado de su garantía y las expectativas respecto a una posible herramienta digital de apoyo.

Preguntas guía utilizadas en las entrevistas:

1. ¿Cómo se registra actualmente una venta y su garantía correspondiente?
2. ¿Qué inconvenientes surgen con mayor frecuencia al momento de responder consultas de clientes sobre el estado de su garantía?
3. ¿Existen distintos períodos de garantía según el tipo de producto? ¿Cómo se determinan?
4. ¿Qué información sería más útil tener disponible de forma rápida al momento de atender a un cliente?
5. ¿Qué mejoras esperarían de un sistema que digitalice este proceso?

Conclusiones de la elicitación:

- El registro actual se realiza en planillas físicas o electrónicas no centralizadas, lo que genera pérdida de tiempo al buscar información de ventas anteriores.
- Los períodos de garantía varían según la categoría del producto (por ejemplo, línea blanca, pequeños electrodomésticos, tecnología), por lo que el sistema debe contemplar esta diferenciación al momento de calcular la vigencia.

- Existe una necesidad concreta de poder consultar rápidamente, ante una consulta del cliente, si la garantía de un producto sigue vigente.
- Se espera que el sistema sea simple de utilizar por el personal de ventas, sin requerir capacitación técnica avanzada.

Tecnologías utilizadas por el cliente (TIC): actualmente, PuntoTecno utiliza planillas de cálculo (Excel) para el registro de ventas y no cuenta con un sistema informático propio para la gestión de garantías, lo cual refuerza la necesidad del desarrollo propuesto.

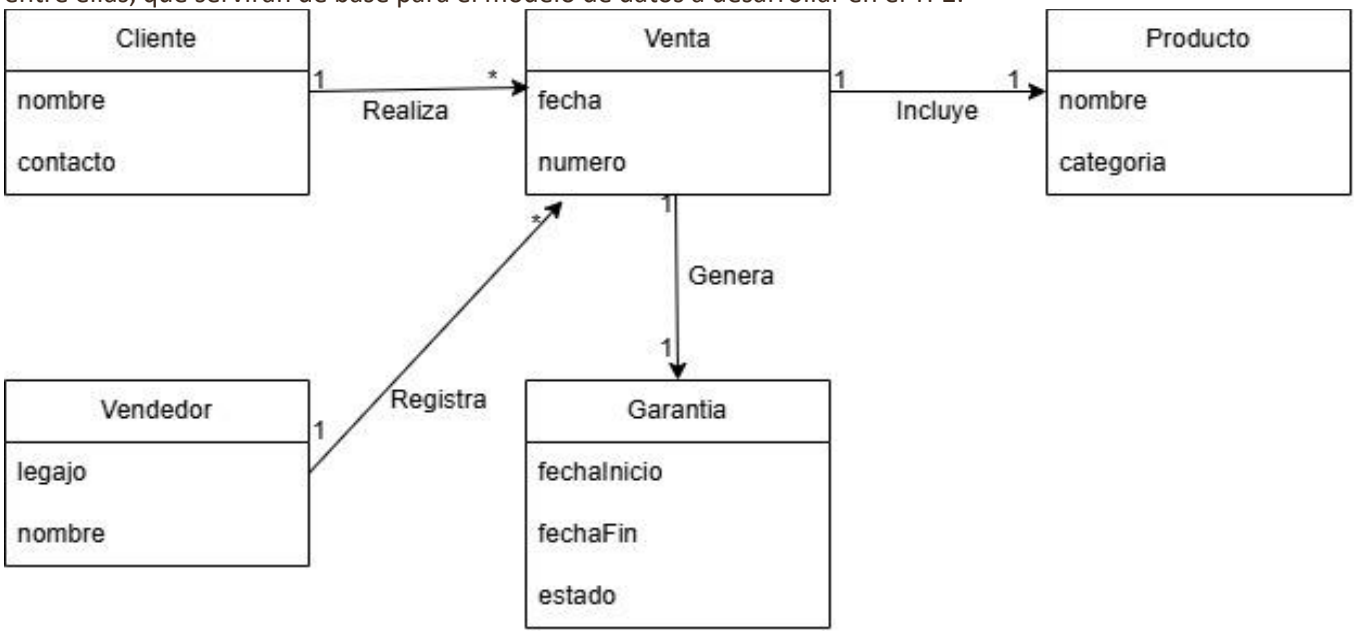
Análisis de competencia: en el rubro de venta de electrodomésticos, algunas cadenas de mayor tamaño (como Garbarino o Musimundo) cuentan con sistemas propios de gestión que permiten a sus empleados consultar el estado de garantía de un producto de forma inmediata a través de sistemas internos. Contar con una herramienta similar, adaptada a la escala de PuntoTecno, permitiría equiparar la calidad de atención frente a competidores de mayor envergadura.

Conocimiento del negocio

El proceso de venta en PuntoTecno consiste en la comercialización de electrodomésticos de distintas categorías (línea blanca, pequeños electrodomésticos y tecnología del hogar), cada una con un período de garantía propio, generalmente definido por el fabricante y respaldado por la política comercial del local.

Cuando un cliente adquiere un producto, el vendedor registra la operación indicando el producto vendido, los datos del cliente y la fecha de la venta. A partir de esa venta, se genera automáticamente una garantía cuya duración depende de la categoría a la que pertenece el producto adquirido (por ejemplo, la línea blanca suele contar con garantías más extensas que los pequeños electrodomésticos). Dicha garantía queda vigente durante el período correspondiente a partir de la fecha de venta, y su estado (vigente o vencida) puede ser consultado en cualquier momento, tanto por el personal del local como, eventualmente, por el propio cliente.

Este conocimiento del negocio resulta clave para el diseño del sistema, ya que permite identificar con claridad las entidades centrales del dominio (Cliente, Vendedor, Producto, Venta y Garantía) y las relaciones que existen entre ellas, que servirán de base para el modelo de datos a desarrollar en el TP2.



Relevamiento de procesos de negocio

Proceso: Registrar venta

- **Roles:** Cliente, Vendedor.
- **Pasos:**
    1. El cliente selecciona el producto que desea comprar en el local.
    2. El vendedor consulta el precio del producto.
    3. El vendedor registra manualmente en una planilla los datos de la venta: producto, cliente y fecha.

4. El vendedor calcula "a mano" el período de garantía correspondiente, según su propio conocimiento de la categoría del producto.
5. El vendedor entrega al cliente un comprobante de venta.

Proceso: Consultar estado de garantía

- **Roles:** Cliente, Vendedor.
- **Pasos:**
    1. El cliente se acerca al local o llama para consultar si su producto sigue en garantía.
    2. El vendedor busca en las planillas históricas la venta correspondiente.
    3. El vendedor calcula manualmente, en base a la fecha de venta y la categoría del producto, si la garantía sigue vigente.
    4. El vendedor informa el resultado al cliente.

Diagnóstico de los procesos relevados

Proceso: Registrar venta

- Problema/s: Registro disperso y no centralizado de las ventas; cálculo manual y no estandarizado del período de garantía.
- Causa/s: Ausencia de un sistema informático que centralice la información; el cálculo de garantía depende del criterio y la memoria de cada vendedor, sin una regla fija y verificable.

Proceso: Consultar estado de garantía

- Problema/s: Demoras al momento de responder a un cliente sobre la vigencia de su garantía; posibilidad de error humano al calcular manualmente si la garantía sigue vigente.
- Causa/s: La información de ventas se encuentra en planillas históricas de difícil búsqueda; no existe una regla automática ni un sistema que determine la vigencia de forma instantánea.

Estos diagnósticos son justamente los que fundamentan la necesidad del sistema: automatizar el registro de la venta y el cálculo/consulta de la garantía para eliminar ambos problemas.

Propuesta de solución

Propuesta funcional

Para dar respuesta a los problemas identificados en el diagnóstico y favorecer el modelo de negocio de PuntoTecno, se propone desarrollar un sistema de gestión de ventas y garantías que permita centralizar el registro de las operaciones comerciales del local y automatizar por completo el cálculo y la consulta del estado de garantía de cada producto vendido.

El sistema permitirá al vendedor registrar cada venta indicando el producto, el cliente y la fecha de la operación. A partir de esos datos, el sistema calculará automáticamente la fecha de vencimiento de la garantía en función de la categoría del producto, eliminando por completo el cálculo manual. Además, permitirá consultar en cualquier momento el estado de vigencia de la garantía de un producto (vigente o vencida), tanto por número de venta como por cliente, agilizando significativamente los tiempos de respuesta al momento de atender una consulta.

Propuesta técnica

El desarrollo del sistema de gestión de ventas y garantías se realizará en Java, lenguaje que cuenta con amplio soporte de bibliotecas y frameworks que facilitan el escalamiento futuro del sistema. La persistencia de los datos se llevará a cabo en MySQL, base de datos relacional abierta, flexible y de alto rendimiento, adecuada para el volumen de operaciones que maneja el local.

Para la conexión entre la aplicación Java y la base de datos se empleará JDBC (Java Database Connectivity), API estándar que permite a las aplicaciones Java interactuar con bases de datos relacionales, realizando consultas, inserciones y actualizaciones de manera directa.

En cuanto a la arquitectura de despliegue, al tratarse de un sistema interno de uso exclusivo del personal del local, se propone lo siguiente:

1. La aplicación Java se instalará en la computadora o computadoras utilizadas por el personal de venta dentro del local, conectadas mediante una red local (LAN) con cableado Ethernet o Wi-Fi.
2. El servidor de base de datos MySQL se alojará en un único equipo dentro del mismo local (o en un servidor local dedicado), accesible por todas las terminales de venta a través de la red interna.

3. Este modelo es escalable ante un eventual crecimiento del negocio, ya sea agregando nuevas terminales de venta dentro del mismo local o, en una instancia posterior, contemplando la conexión remota entre sucursales si el negocio se expandiera.

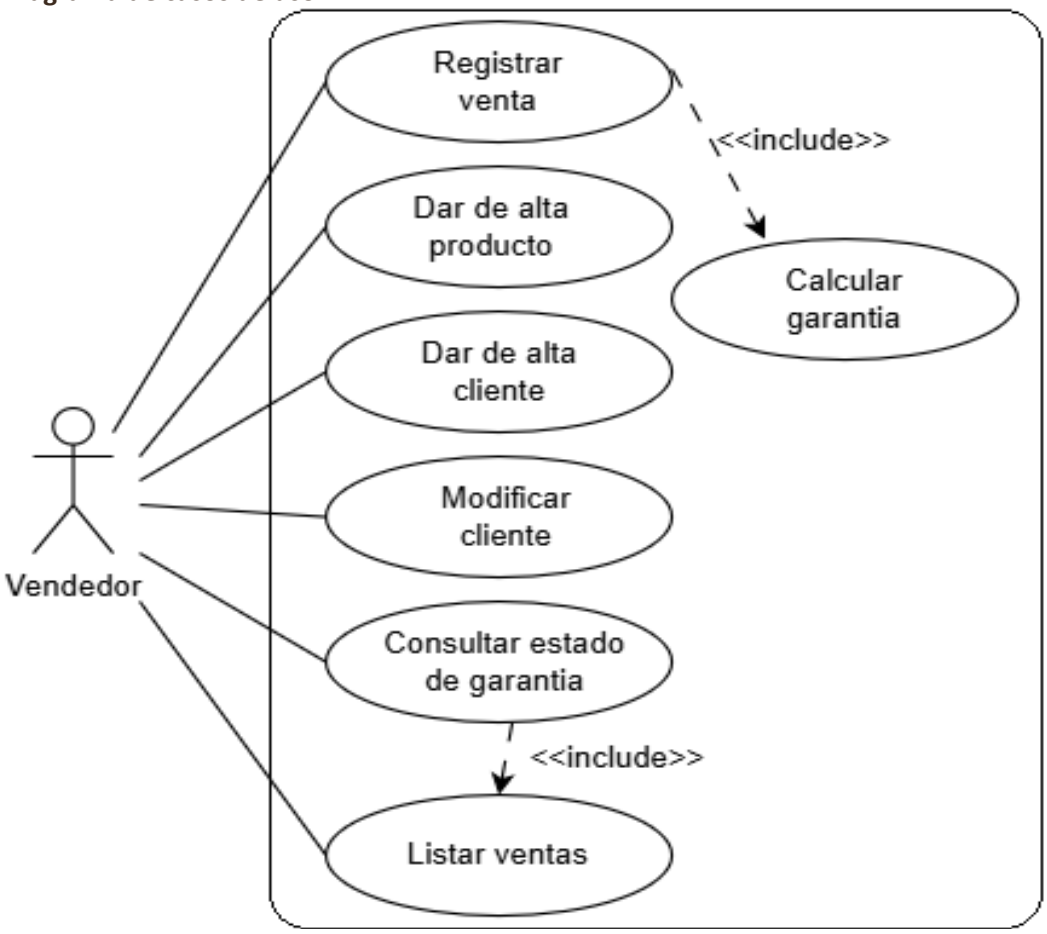
Requerimientos funcionales

Requerimiento de Sistema	Descripción
RFS01	El sistema debe permitir el ingreso mediante un usuario y contraseña.
RFS02	El sistema debe permitir registrar una nueva venta, indicando cliente, producto, vendedor y fecha.
RFS03	Al registrar una venta, el sistema debe calcular automáticamente la fecha de vencimiento de la garantía según la categoría del producto.
RFS04	El sistema debe permitir dar de alta un nuevo producto, indicando nombre, categoría y precio.
RFS05	El sistema debe permitir dar de alta un nuevo cliente, indicando nombre y datos de contacto.
RFS06	El sistema debe permitir consultar el estado de garantía (vigente o vencida) de un producto vendido.
RFS07	El sistema debe permitir consultar el historial de ventas realizadas a un cliente determinado.
RFS08	El sistema debe permitir consultar el historial de ventas y garantías asociadas a un producto determinado.
RFS09	El sistema debe permitir listar todas las ventas registradas, con la posibilidad de filtrarlas por fecha.
RFS10	El sistema debe permitir modificar los datos de un cliente ya registrado.

Requerimientos no funcionales

Requerimiento de Sistema	Descripción
RNF01	El sistema debe estar desarrollado en Java.
RNF02	El sistema debe contar con una base de datos MySQL.
RNF03	El sistema debe utilizar JDBC (Java Database Connectivity) para la conexión entre la aplicación y la base de datos.
RNF04	El sistema debe tener disponibilidad durante el horario comercial del local (lunes a sábado, 9 a 20 hs).
RNF05	El sistema debe ser escalable, permitiendo la incorporación de nuevas terminales de venta dentro del local.
RNF06	Para la persistencia y consulta de datos en la base de datos se debe utilizar un patrón MVC (modelo-vista-controlador).
RNF07	El sistema debe estar instalado en la infraestructura propia de PuntoTecno.
RNF08	El tiempo de respuesta ante una consulta de estado de garantía no debe superar los 3 segundos.

Diagrama de casos de uso



Identificación de actores

- Vendedor: persona humana que trabaja en PuntoTecno y está autorizada a registrar ventas, dar de alta productos y clientes, y consultar el estado de garantías utilizando el sistema.

Trazabilidad

Requerimiento	Caso de uso relacionado
RFS02	Registrar venta
RFS03	Calcular garantía
RFS04	Dar de alta producto
RFS05	Dar de alta cliente
RFS06	Consultar estado de garantía
RFS07	Listar ventas *(consulta por cliente)*
RFS08	Listar ventas *(consulta por producto)*
RFS09	Listar ventas
RFS10	Modificar cliente
RFS06	Todos los casos de uso *(implementación con patrón MVC)*

Descripción de casos de uso

Se describirán 2 casos de uso a modo de ejemplo:

Caso de uso: Registrar venta

Campo	Descripción
Actor principal	Vendedor
Precondición	El vendedor debe estar autenticado en el sistema; el cliente y el producto deben existir previamente registrados.
Flujo principal	1. El vendedor selecciona la opción "Registrar venta". 2. El vendedor selecciona el cliente y el producto de la venta. 3. El vendedor confirma la fecha de la operación. 4. El sistema ejecuta el caso de uso incluido "Calcular garantía", determinando la fecha de vencimiento según la categoría del producto. 5. El sistema persiste la venta y la garantía asociada. 6. El sistema muestra un mensaje de confirmación al vendedor.
Flujo alternativo	Si el cliente o el producto no existen, el sistema informa el error y solicita darlos de alta previamente (casos de uso "Dar de alta cliente" / "Dar de alta producto").
Poscondición	La venta y su garantía correspondiente quedan registradas y disponibles para su consulta.

Caso de uso: Consultar estado de garantía

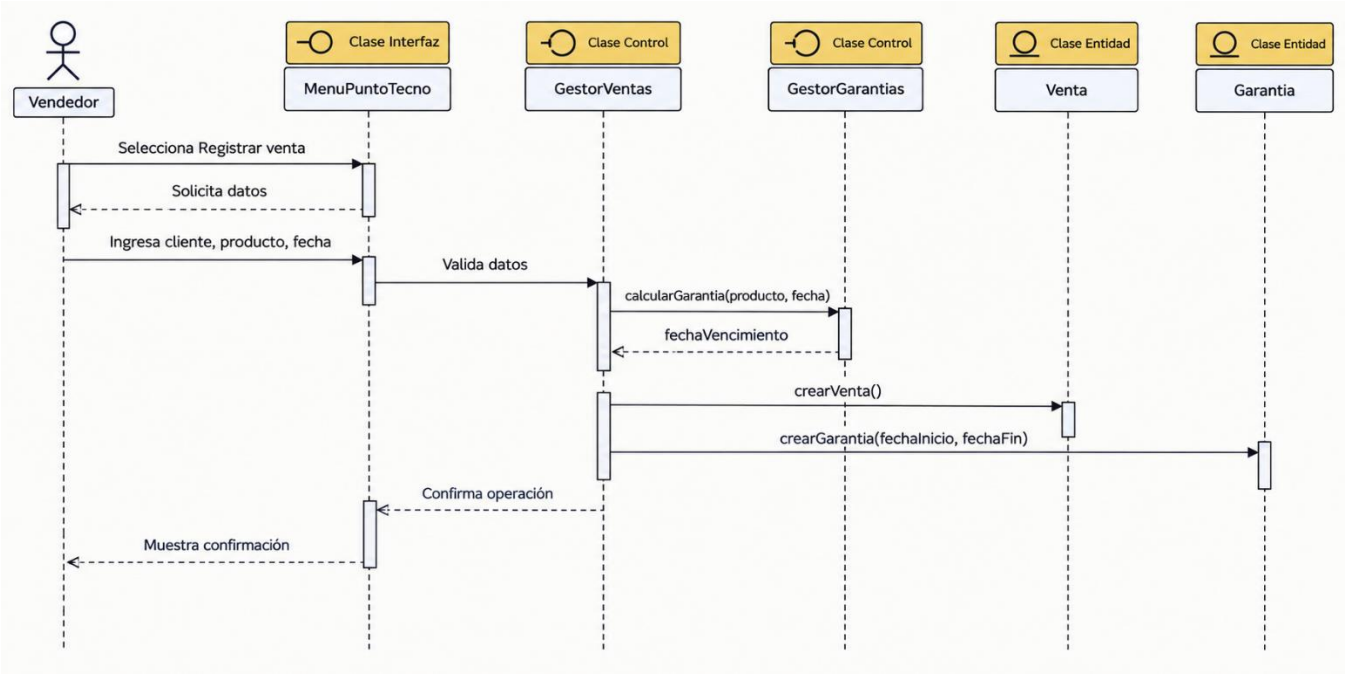
Campo	Descripción
Actor principal	Vendedor
Precondición	El vendedor debe estar autenticado en el sistema; debe existir al menos una venta registrada.
Flujo principal	1. El vendedor selecciona la opción "Consultar estado de garantía". 2. El vendedor ingresa el número de venta o busca por cliente/producto. 3. El sistema obtiene la información correspondiente. 4. El sistema calcula si la fecha actual se encuentra dentro del período de garantía. 5. El sistema muestra al vendedor el estado (vigente o vencida) y la fecha de vencimiento.
Flujo alternativo	Si no se encuentra ninguna venta con los datos ingresados, el sistema informa que no existen resultados.
Poscondición	El vendedor obtiene el estado actualizado de la garantía consultada.

Etapa de análisis

Clases de análisis

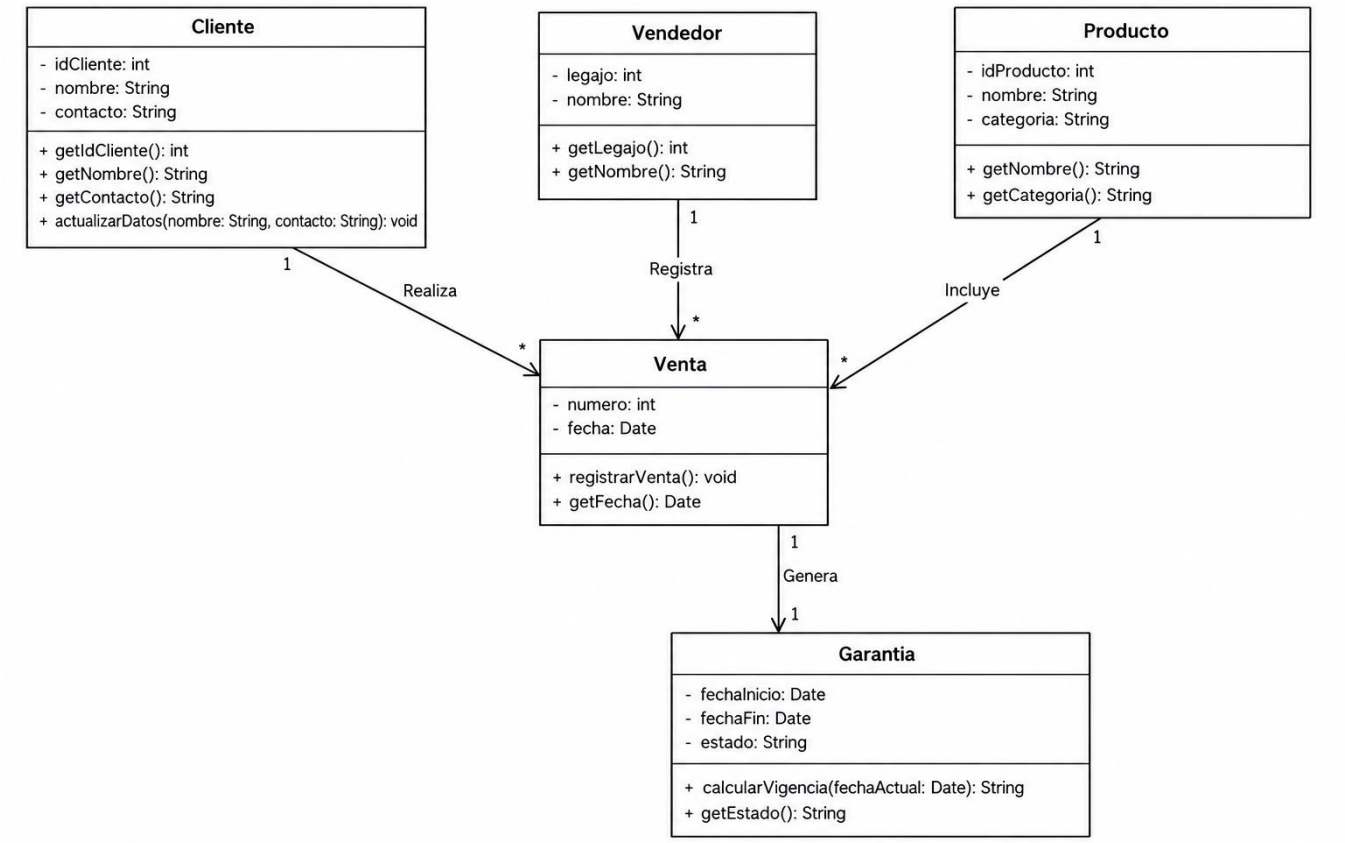
Clase	Estereotipo	Responsabilidad
MenuPuntoTecno	Interfaz	Muestra el menú principal al vendedor y captura su selección (Registrar venta, Dar de alta producto, Dar de alta cliente, Modificar cliente, Consultar estado de garantía, Listar ventas, Salir).
GestorVentas	Control	Coordina el flujo de registro de venta: valida datos, invoca el cálculo de garantía y ordena la persistencia.
GestorGarantias	Control	Calcula la fecha de vencimiento según categoría del producto y determina el estado (vigente/vencida) ante una consulta.
Cliente	Entidad	Conoce sus propios datos (nombre, contacto) y los expone/actualiza.
Vendedor	Entidad	Conoce legajo y nombre; queda asociado a la venta que registra.
Producto	Entidad	Conoce nombre y categoría (de la categoría depende el período de garantía).
Venta	Entidad	Conoce fecha, número, y sus relaciones con Cliente, Vendedor y Producto.
Garantía	Entidad	Conoce fechaInicio, fechaFin y estado; sabe calcular si está vigente.

Diagrama de secuencia – Caso de uso “Registrar venta”



Este diagrama muestra el flujo del **CU001 "Registrar venta"**, incluyendo el CU incluido "Calcular garantía": el vendedor carga los datos, GestorVentas delega en GestorGarantias el cálculo de la fecha de vencimiento según la categoría del producto, y recién ahí se crean los objetos Venta y Garantía.

Etapa de diseño



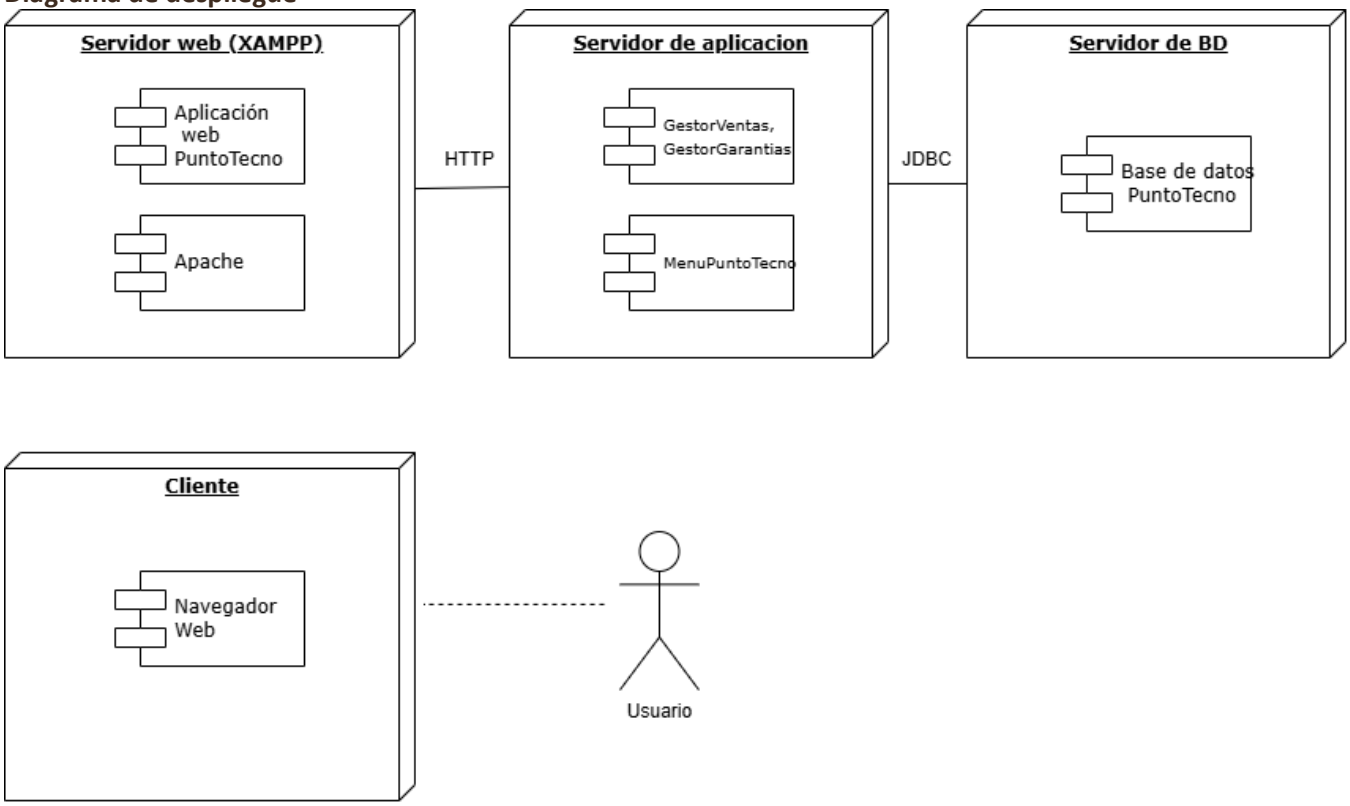
Justificación de las relaciones y métodos:

- Venta.registrarVenta() es el método que internamente coordina la asociación con Cliente, Vendedor y Producto, y dispara la creación de la Garantia.
- Garantia.calcularVigencia() recibe la fecha actual y determina si sigue vigente comparando contra fechaFin (esto responde al RFS06).
- Las visibilidades (- privado para atributos, + público para métodos) siguen la convención estándar de UML.

Etapa de implementación

Para la etapa de implementación se mantienen los requerimientos ya definidos: el sistema se desarrolla en **Java (RNF01)** y persiste los datos en **MySQL (RNF02)**. Para poner en marcha este entorno de forma local en PuntoTecno, se utiliza **XAMPP** como entorno de desarrollo web de código abierto, ya que integra Apache (que cumple la función de servidor web y de aplicación) junto con MySQL en una sola instalación. Esto simplifica la configuración, no requiere licencias ni infraestructura adicional, y permite alojar el sistema dentro de la red propia del local, cumpliendo así con lo ya establecido sin sumar restricciones nuevas.

Diagrama de despliegue



Lectura del diagrama:

- El **Cliente** (vendedor con navegador web) se conecta por **HTTP** al **Servidor web (XAMPP)**, que aloja la aplicación web PuntoTecno sobre Apache.
- El **Servidor web** se comunica por **HTTP** con el **Servidor de aplicación**, donde residen las clases de control (GestorVentas, GestorGarantias) y la interfaz (MenuPuntoTecno).
- El **Servidor de aplicación** se conecta por **JDBC** al **Servidor de base de datos MySQL**, que persiste la base puntotecno (con las tablas de clientes, vendedores, productos, ventas y garantías).

Etapa de pruebas

Plan de pruebas

El plan de pruebas para el caso de uso **"Consultar estado de garantía"** contempla dos niveles: una prueba de componente sobre el método que resuelve la lógica más sensible del sistema (calcularVigencia() de la clase Garantía), aplicando la técnica de caja blanca por valores límite; y una prueba de sistema sobre el caso de uso completo, verificando el comportamiento de punta a punta desde la perspectiva del vendedor (caja negra). El criterio de aceptación es que, en todos los casos, el estado informado (vigente/vencida) coincida exactamente con el esperado según la fecha de vencimiento calculada.

Caso de uso	Código prueba	Tipo de prueba	Técnica propuesta	Observaciones
Consultar estado de garantía	CPC01	Componente	Cobertura por valores límite (caja blanca)	Aplicación de caja blanca al método calcularVigencia() de la clase Garantía, probando los límites de vigencia según el período de garantía de cada categoría de producto.
Consultar estado de garantía	CPS01	Sistema	Funcional (caja negra)	Verificación del flujo completo del caso de uso: ingreso del número de venta, obtención de la garantía asociada y visualización correcta del estado al vendedor.

Casos de prueba:

CPC01 (valores límite sobre calcularVigencia())

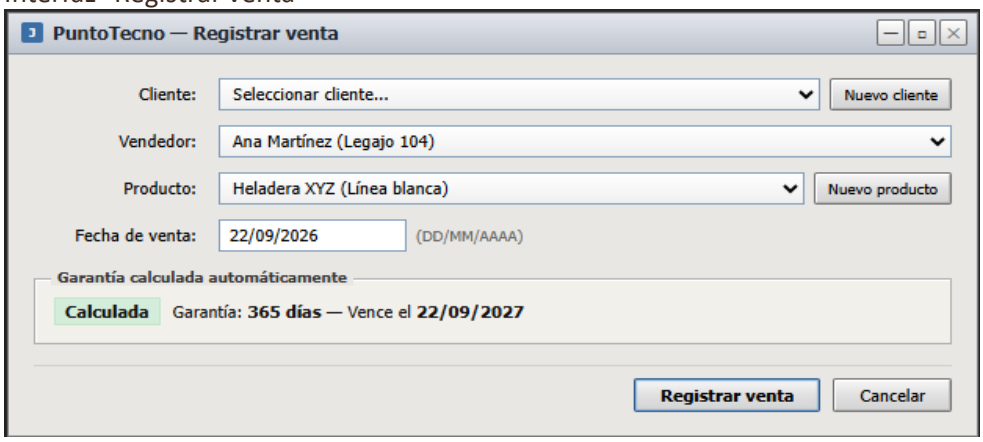
N° caso	Categoría	Periodo garantía	Fecha vencimiento	Fecha de consulta	Resultado esperado	Observación
CP01	Línea blanca	365 días	01/01/2027	31/12/2026	Vigente	Un día antes del vencimiento
CP02	Línea blanca	365 días	01/01/2027	01/01/2027	Vigente	Día exacto del vencimiento
CP03	Línea blanca	365 días	01/01/2027	02/01/2027	Vencida	Un día después del vencimiento
CP04	Pequeños electrodomésticos	180 días	30/06/2026	29/06/2026	Vigente	Un día antes del vencimiento
CP05	Pequeños electrodomésticos	180 días	30/06/2026	30/06/2026	Vigente	Día exacto del vencimiento
CP06	Pequeños electrodomésticos	180 días	30/06/2026	01/07/2026	Vencida	Un día después del vencimiento
CP07	Tecnología del hogar	90 días	01/04/2026	31/03/2026	Vigente	Un día antes del vencimiento
CP08	Tecnología del hogar	90 días	01/04/2026	01/04/2026	Vigente	Día exacto del vencimiento
CP09	Tecnología del hogar	90 días	01/04/2026	02/04/2026	Vencida	Un día después del vencimiento

CPS01 (funcional, caja negra sobre el caso de uso)

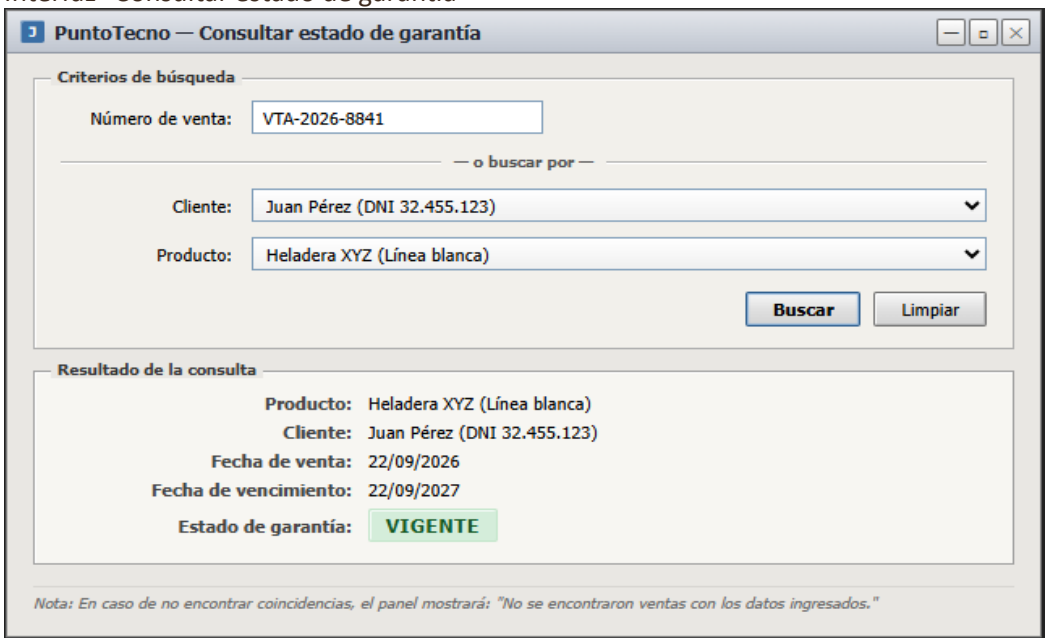
N° caso	Entrada (número de venta)	Escenario	Resultado esperado
CP10	Número de venta existente, garantía vigente	El vendedor consulta una venta reciente	El sistema muestra estado "Vigente" y la fecha de vencimiento
CP11	Número de venta existente, garantía vencida	El vendedor consulta una venta antigua	El sistema muestra estado "Vencida" y la fecha en que venció
CP12	Número de venta inexistente	El vendedor ingresa un número que no está registrado	El sistema informa que no existen resultados
CP13	Búsqueda por cliente con varias ventas	El vendedor busca por nombre de cliente	El sistema lista todas las ventas y garantías asociadas a ese cliente

Interfaz grafica

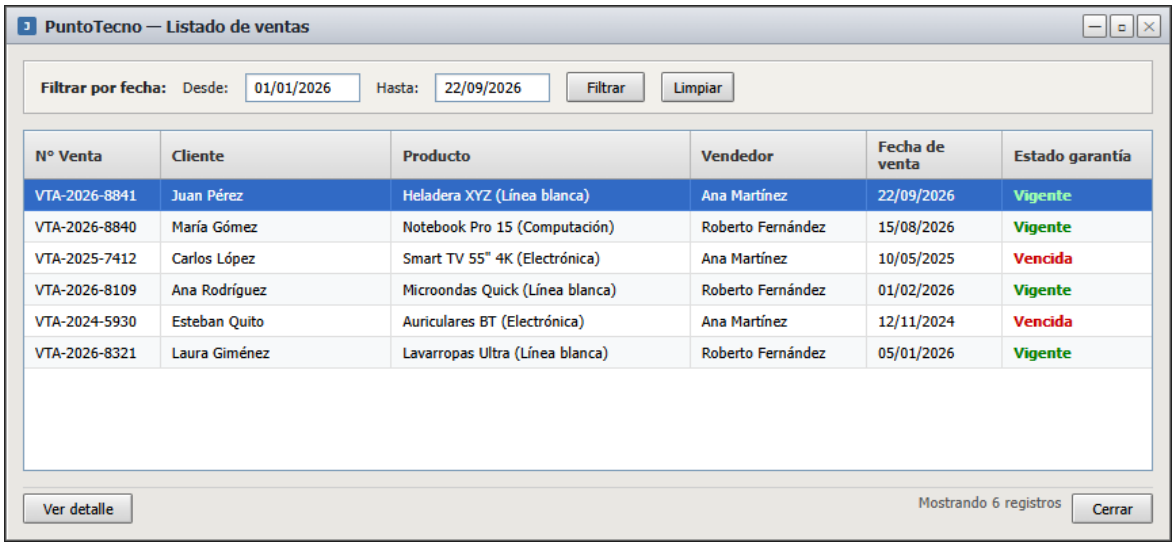
Interfaz "Registrar venta"



Interfaz "Consultar estado de garantía"



Interfaz "Listar ventas"



Justificación de la vinculación interfaz-caso de uso:

Prototipo 1 — Registrar venta: esta interfaz materializa directamente el caso de uso **"Registrar venta"** (y su incluido "Calcular garantía"). Cada campo del formulario corresponde a un paso del flujo principal definido en el TP1: la selección de cliente, vendedor y producto responde al paso 2 ("El vendedor selecciona el cliente y el producto de la venta"), la confirmación de fecha al paso 3, y el panel de garantía calculada automáticamente representa en pantalla el resultado del paso 4, donde el sistema ejecuta "Calcular garantía" sin intervención manual del vendedor. Los botones "Nuevo cliente" y "Nuevo producto" cubren el flujo alternativo ("Si el cliente o el producto no existen, el sistema informa el error y solicita darlos de alta previamente"), evitando que el vendedor deba salir de la pantalla para completar la operación.

Prototipo 2 — Consultar estado de garantía: esta interfaz implementa el caso de uso **"Consultar estado de garantía"**. Las dos vías de búsqueda (por número de venta o por cliente/producto) responden al paso 2 del flujo principal ("El vendedor ingresa el número de venta o busca por cliente/producto"), y el panel de resultado con el estado destacado en verde o rojo resuelve visualmente el paso 5 ("El sistema muestra al vendedor el estado — vigente o vencida — y la fecha de vencimiento"). El mensaje de "no se encontraron ventas" cubre el flujo alternativo del caso de uso, asegurando que el vendedor tenga una respuesta clara incluso ante un dato inexistente.

Prototipo 3 — Listar ventas: esta interfaz responde al caso de uso **"Listar ventas"**, que en el TP1 incluye al caso de uso "Consultar estado de garantía" para poder mostrar, además del historial, la vigencia de cada garantía en la misma vista. El filtro por fecha corresponde al RFS09 ("El sistema debe permitir listar todas las ventas registradas, con la posibilidad de filtrarlas por fecha"), y la columna de estado de garantía en la tabla resuelve al mismo tiempo los RFS07 y RFS08 (consulta del historial tanto por cliente como por producto), ya que desde esta única pantalla el vendedor puede identificar de un vistazo la vigencia de todas las garantías asociadas a un cliente o producto determinado.

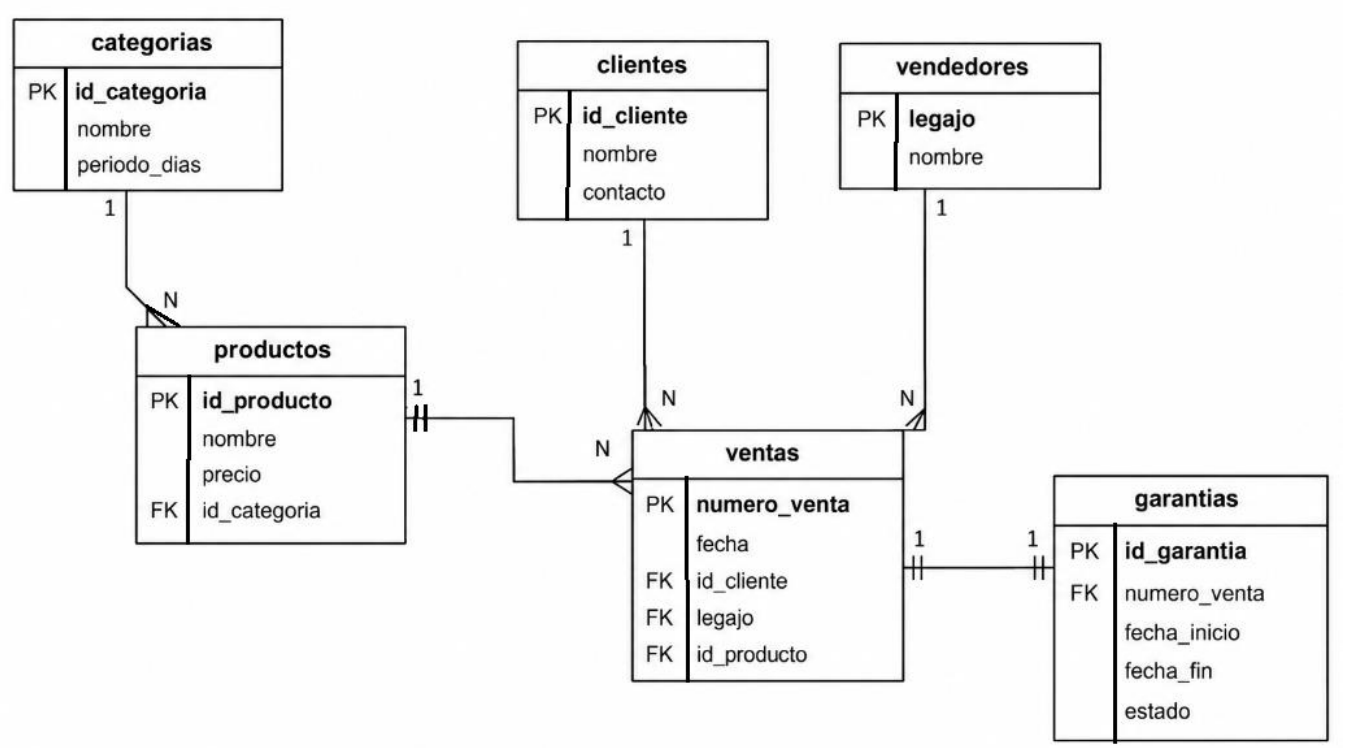
TP 2

Diagrama entidad-relación

Estructura de la tabla

Tabla	Descripción
Categorías	Identificador único, nombre de la categoría (línea blanca, pequeños electrodomésticos, tecnología) y el período de garantía en días correspondiente.
Productos	Identificador único, nombre, precio y la categoría a la que pertenece (FK).
Clientes	Identificador único, nombre y contacto.
Vendedores	Legajo (PK) y nombre.
Ventas	Número de venta (PK), fecha, y las referencias (FK) al cliente, vendedor y producto involucrados.
Garantías	Identificador único, la venta a la que pertenece (FK, relación 1 a 1), fecha de inicio, fecha de fin y estado.

Diagrama visual



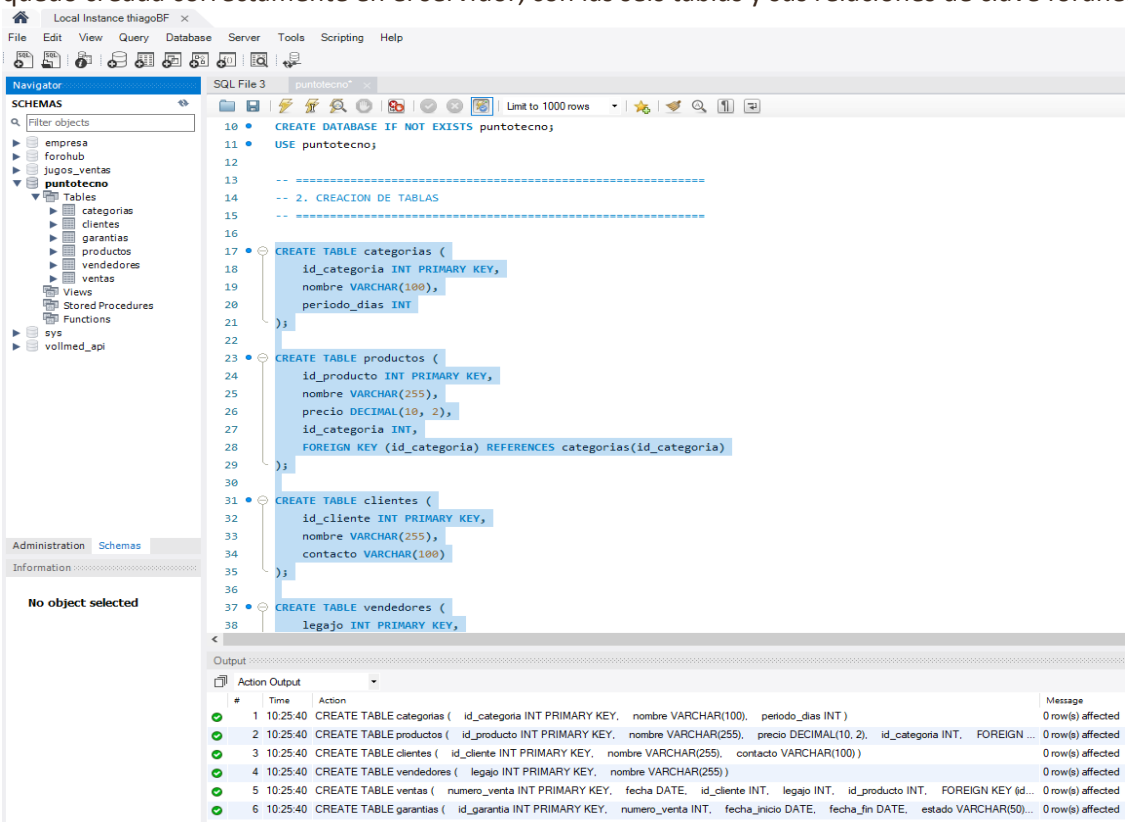
Presentación de las consultas SQL

https://github.com/Thiago-BF/SeminarioDePracticasDeInformatica-Siglo21/blob/main/puntotecno.sql

Evidencia de funcionamiento de la base de datos

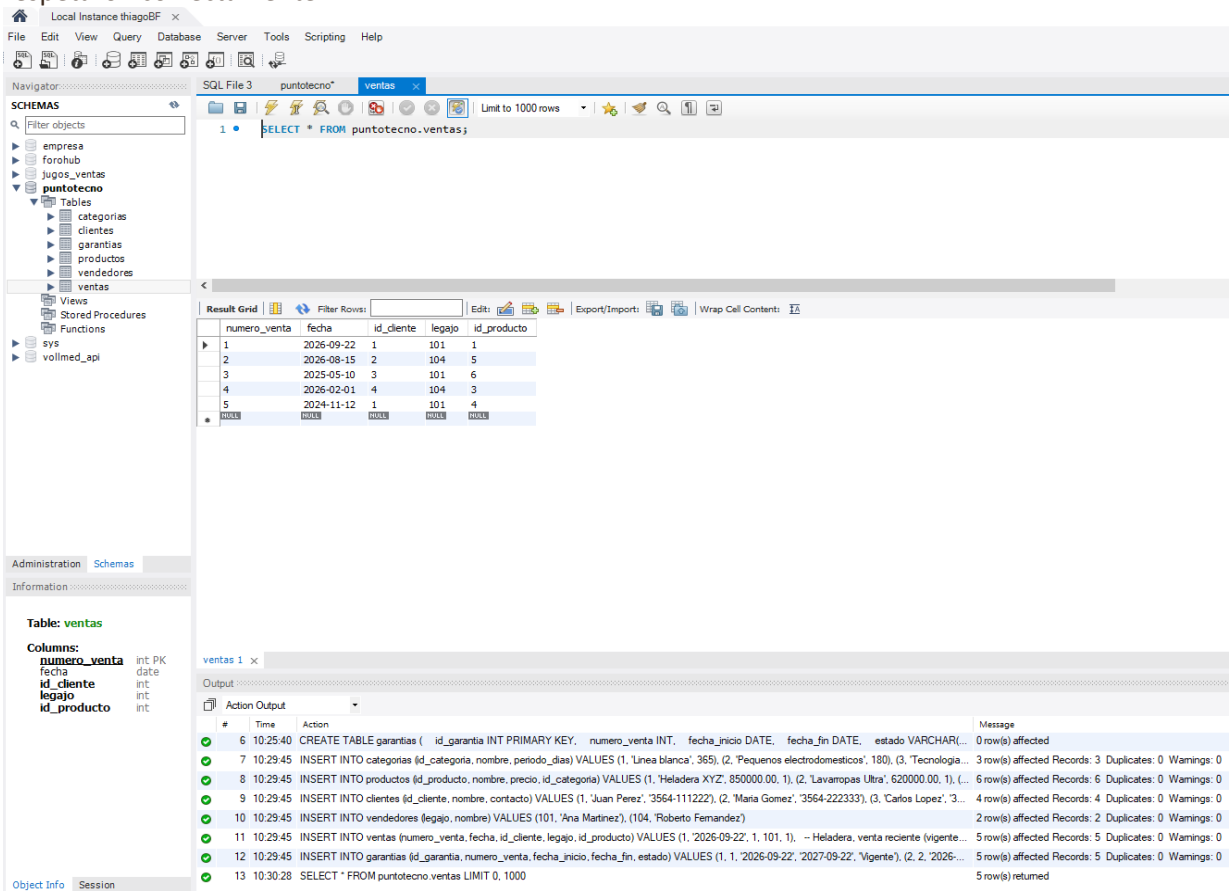
Creación de la base y las tablas:

Se ejecutan las sentencias CREATE DATABASE y CREATE TABLE correspondientes a las seis entidades del modelo (categorías, productos, clientes, vendedores, ventas y garantías). La captura evidencia que la base puntotecno quedó creada correctamente en el servidor, con las seis tablas y sus relaciones de clave foránea ya establecidas.



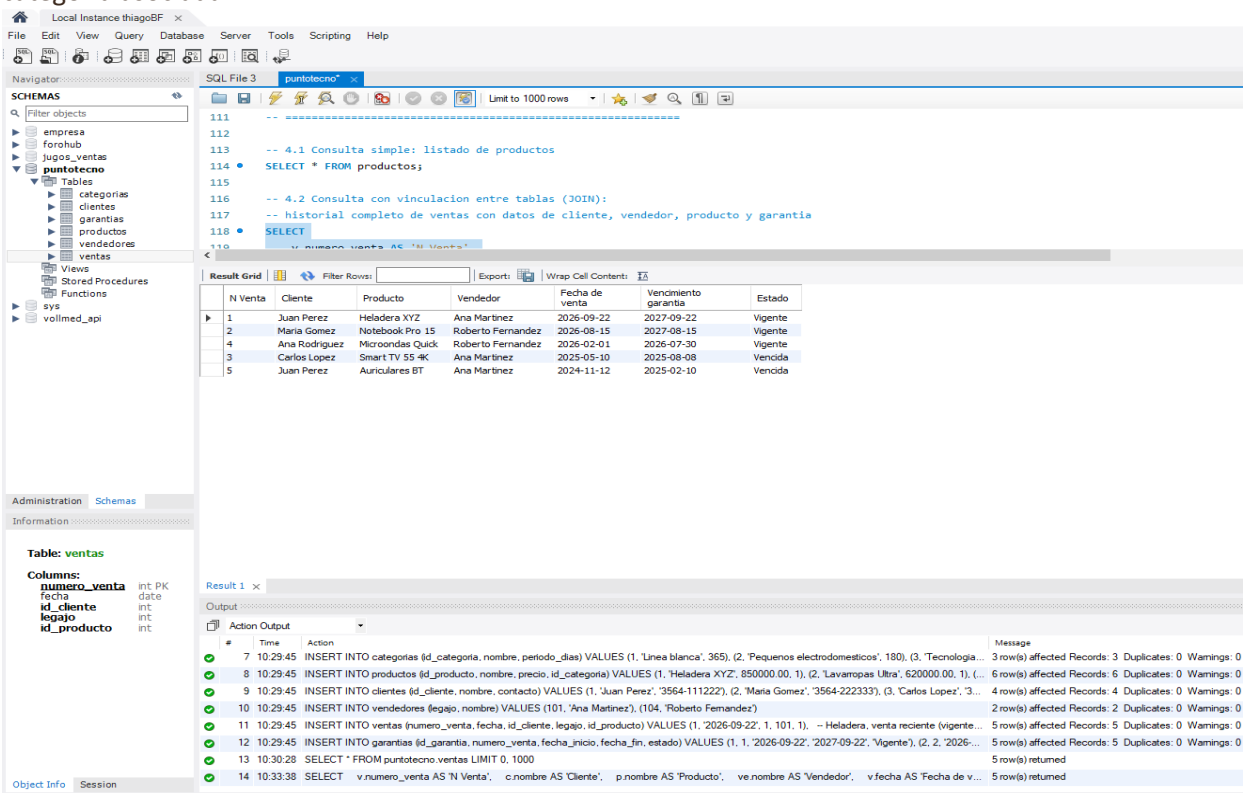
Inserción de datos de prueba:

Se insertan registros de prueba en todas las tablas para simular la operatoria del local. La captura muestra los datos cargados en la tabla ventas, evidenciando que las claves foráneas hacia cliente, vendedor y producto se respetaron correctamente.

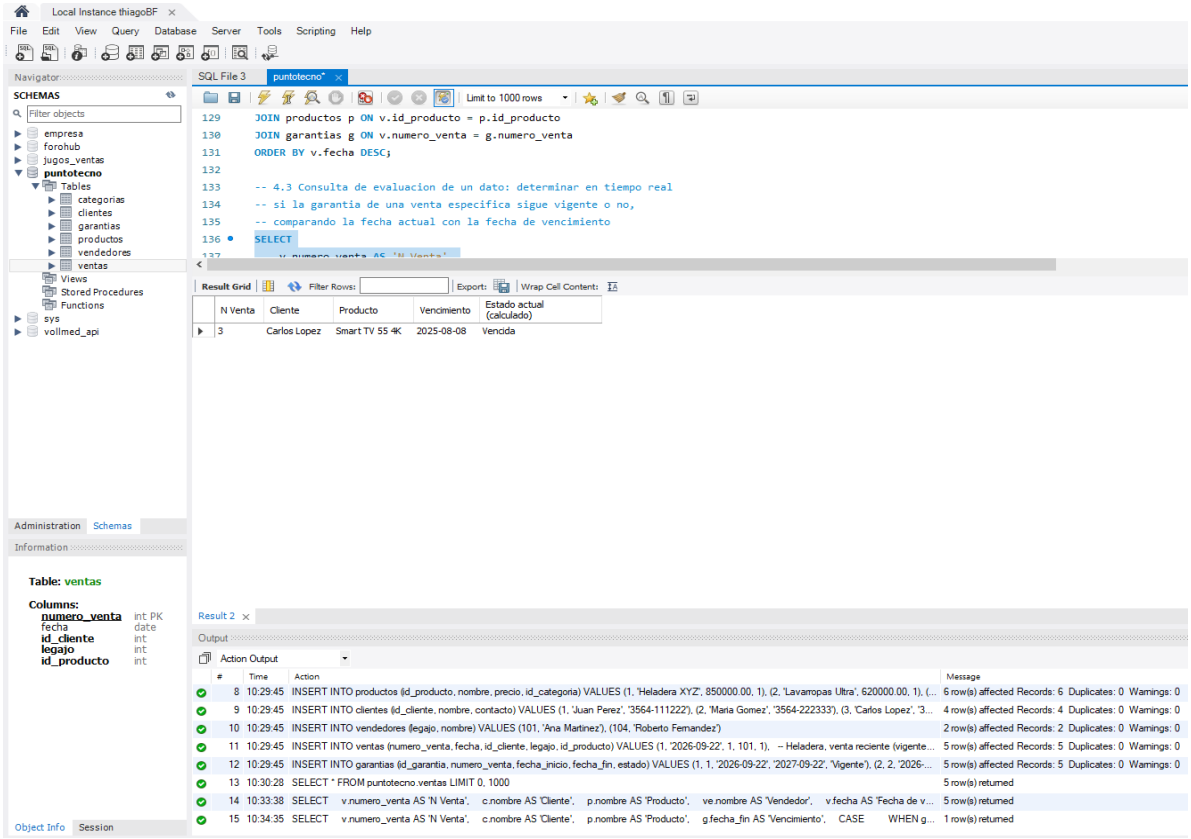


Consulta simple

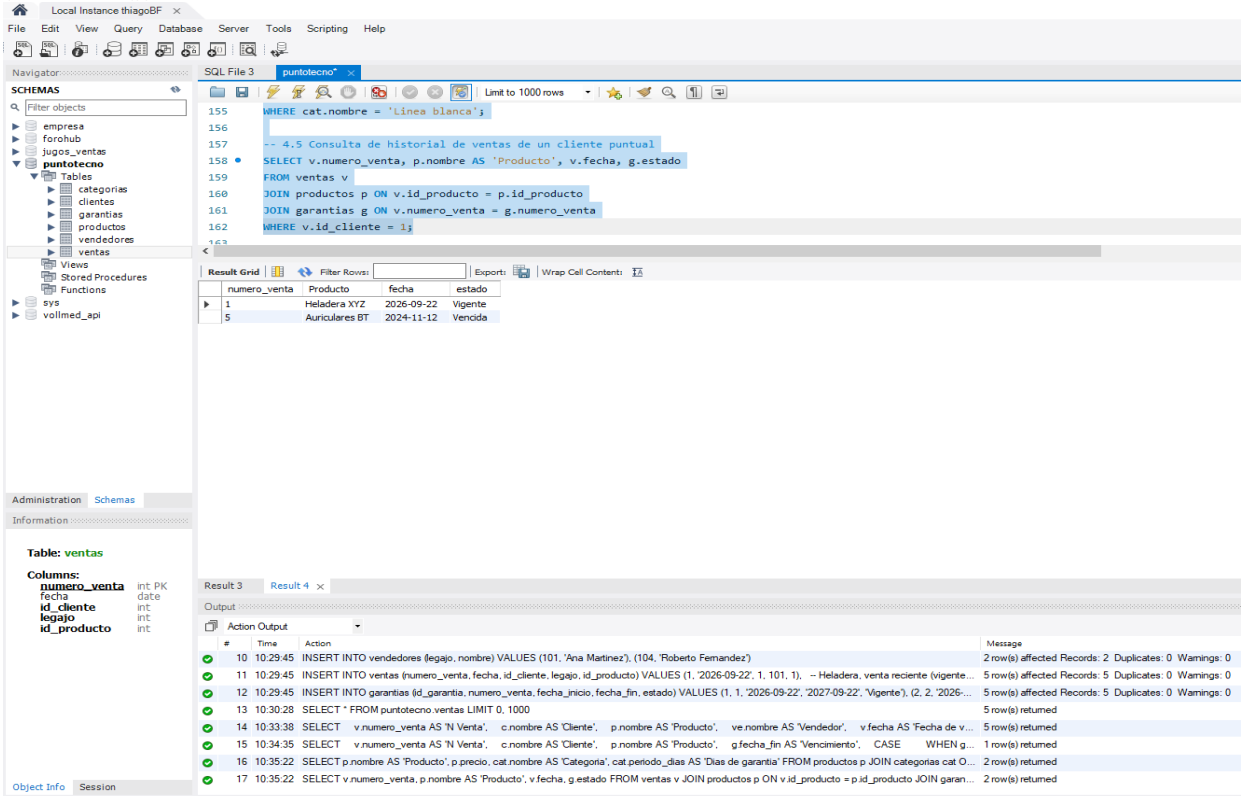
Se ejecuta una consulta SELECT sobre la tabla productos, mostrando el listado completo con nombre, precio y categoría asociada.



Consulta con vinculación entre tablas:
Se ejecuta una consulta que vincula mediante JOIN las tablas ventas, clientes, vendedores, productos y garantías, obteniendo un historial completo de operaciones con el estado de garantía de cada una en un solo resultado.

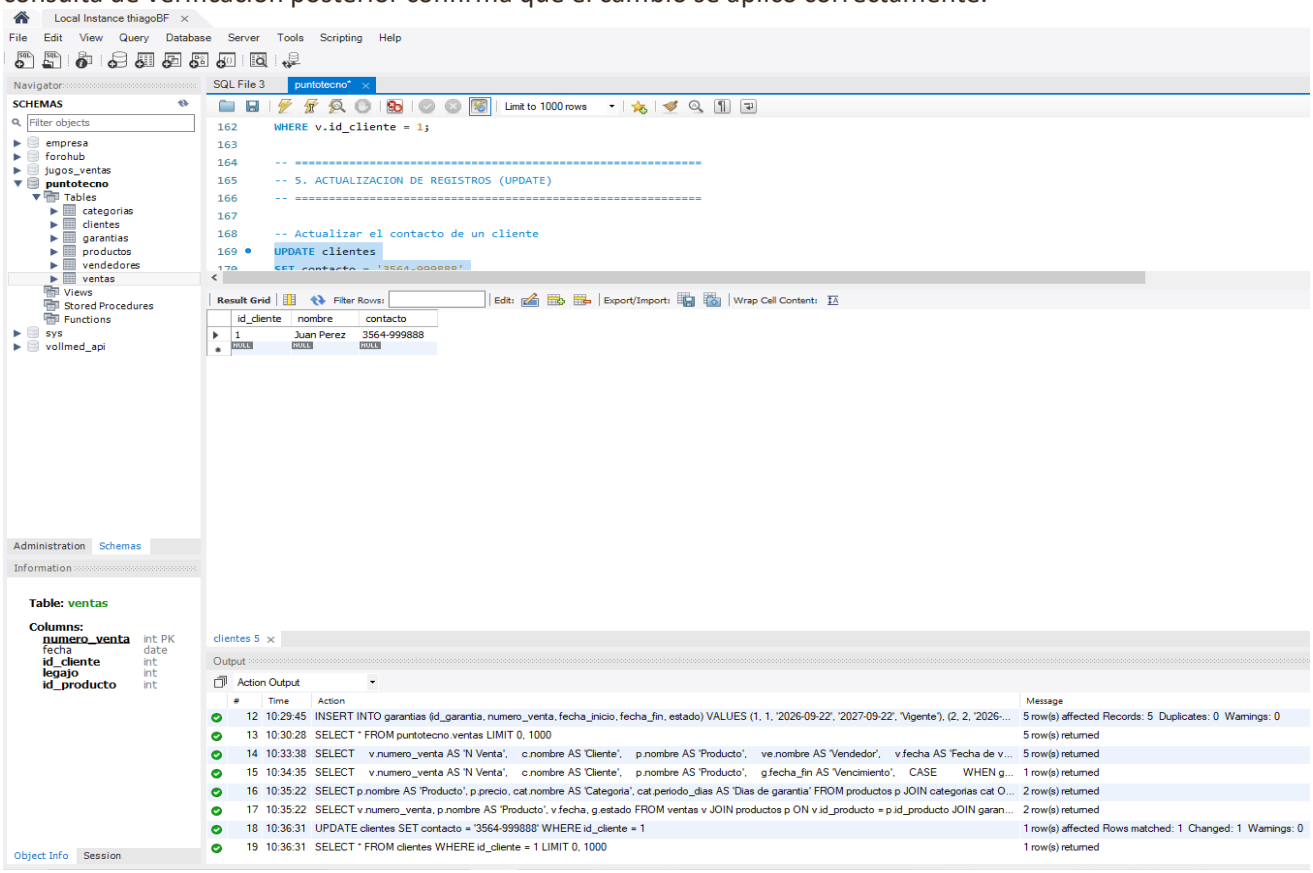


Consulta de evaluación de un dato:
Se ejecuta una consulta que calcula en tiempo real, mediante CASE WHEN, si la garantía de una venta puntual está vigente o vencida, comparando la fecha de vencimiento almacenada contra la fecha actual del sistema (CURDATE()).



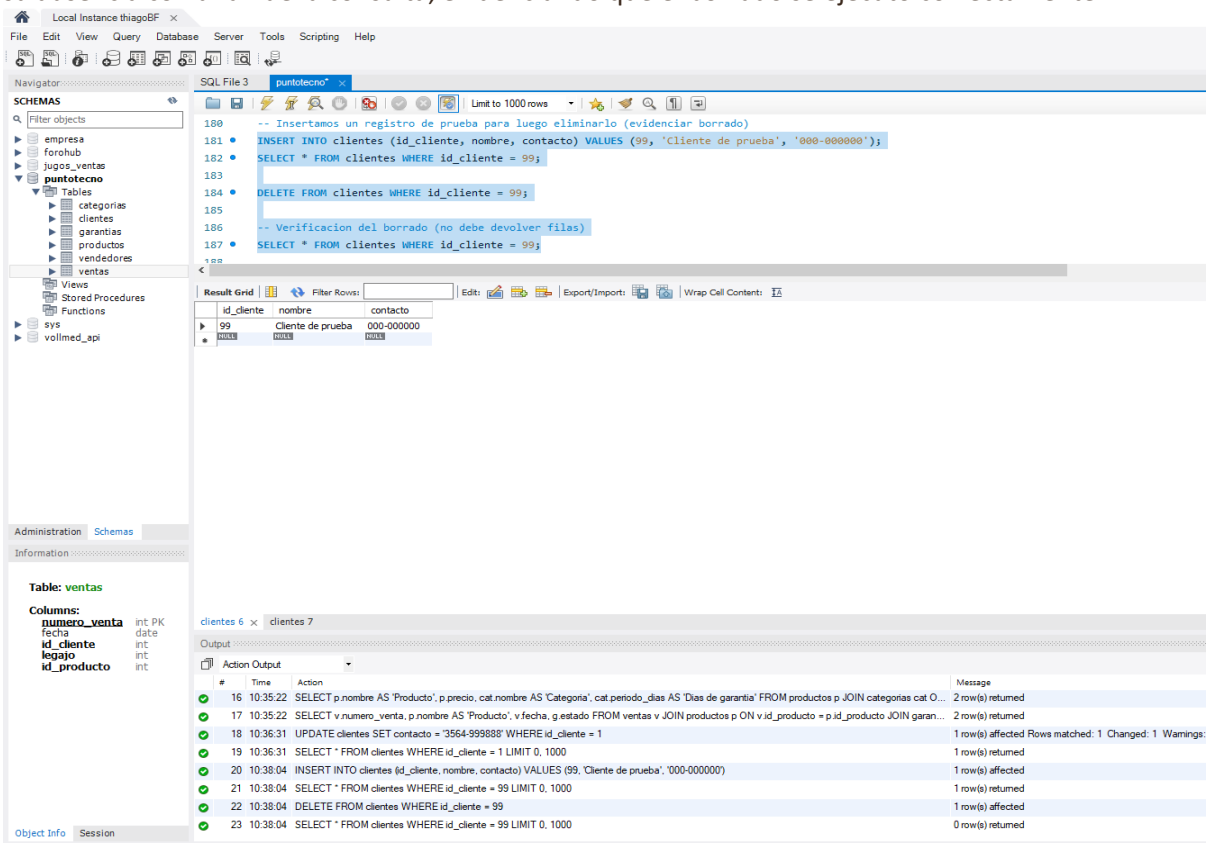
Actualización de un registro:

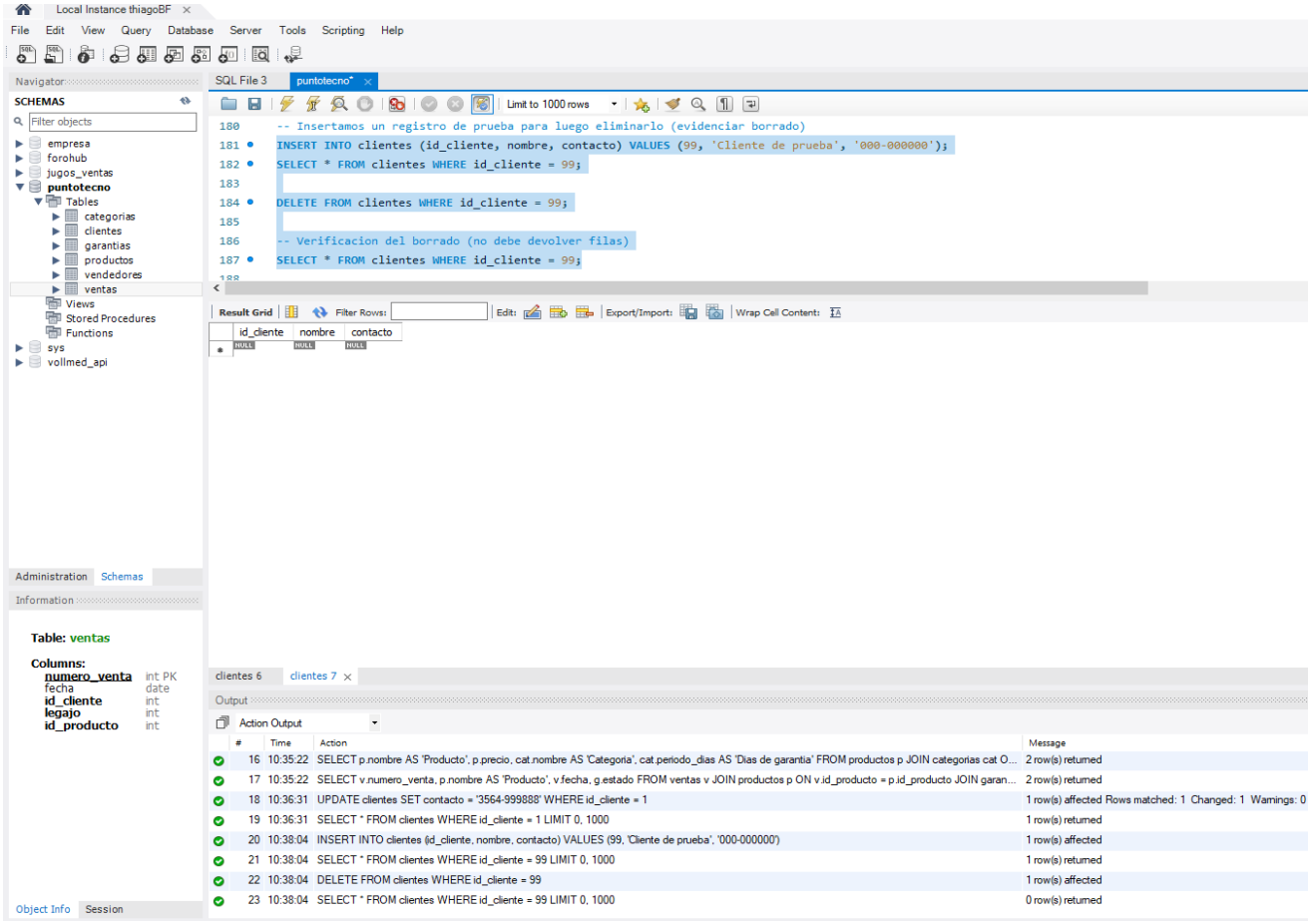
Se ejecuta una sentencia UPDATE sobre la tabla clientes, modificando el dato de contacto de un cliente. La consulta de verificación posterior confirma que el cambio se aplicó correctamente.



Borrado de un registro:

Se inserta un registro de prueba en clientes, se verifica su existencia, se elimina mediante DELETE y se confirma su ausencia con una nueva consulta, evidenciando que el borrado se ejecutó correctamente.





Comunicación

Entorno de red e infraestructura física

El sistema opera dentro de una red de área local (LAN) propia de PuntoTecno, conectando las terminales de venta con el servidor mediante cableado Ethernet o Wi-Fi, sin necesidad de conexión a internet para su funcionamiento habitual. Esta infraestructura es suficiente dado el volumen de operaciones del local y su carácter de sistema interno.

Protocolos de comunicación

- Entre el cliente (navegador web del vendedor) y el servidor web (XAMPP/Apache): protocolo HTTP, sobre el cual viaja la aplicación web de PuntoTecno.
- Entre el servidor de aplicación y el servidor de base de datos MySQL: JDBC (Java Database Connectivity), que permite ejecutar las consultas SQL y gestionar la persistencia de los datos desde el código Java.

Control de enlace de datos

A nivel de enlace, la LAN utiliza el estándar Ethernet (IEEE 802.3) para las conexiones cableadas y Wi-Fi (IEEE 802.11) para las inalámbricas, ambos con control de acceso al medio y direccionamiento por MAC, garantizando la comunicación estable entre las terminales y el servidor dentro del local.

Sistemas externos

En esta primera versión del sistema, PuntoTecno no interactúa con sistemas externos (no se contempla facturación electrónica ni integración con proveedores), por lo que la comunicación queda acotada a los componentes internos ya descriptos: cliente, servidor web, servidor de aplicación y servidor de base de datos.

TP 3

Empezando la elaboración del código

El prototipo es una aplicación de escritorio con interfaz gráfica (Swing) y un menú principal desde el cual se navega a todas las funciones: registrar ventas, dar de alta productos y clientes, modificar clientes, consultar el estado de una garantía y listar ventas. Los datos se mantienen en memoria mediante colecciones ArrayList.

Decisiones técnicas

Aspecto	Decisión	Justificación
Lenguaje	Java (JDK 22)	Restricción del proyecto (RNF01).
Entorno	IntelliJ IDEA	IDE utilizado para escribir, compilar y ejecutar el prototipo.
Interfaz	Swing	Viene incluida en el JDK, no requiere librerías externas. La ventana del menú principal es el punto de navegación de la aplicación.
Datos	ArrayList en memoria	El TP3 no incluye persistencia real. Se precargan los mismos datos de prueba del script puntotecno.sql del TP2.
Estado de la garantía	Se calcula, no se guarda	Se compara la fecha de vencimiento con la fecha actual cada vez que se consulta, como en la consulta 4.3 del TP2 con CURDATE(). Así el estado nunca queda desactualizado.
Base de datos	MySQL (TP4)	La conexión con JDBC se hará en el TP4, llenando las listas desde las tablas existentes.

Explicación del desarrollo en Java

Organización del proyecto

Todo el código está en el paquete puntotecno y se organiza en cuatro grupos de clases, con una responsabilidad clara cada una. Esta separación entre datos (modelo), lógica de gestión e interfaz deja el sistema ordenado y preparado para incorporar el patrón MVC indicado en el RNF06.

Grupo	Clases	Responsabilidad
Modelo	`Producto (abstracta), LineaBlanca, PequenoElectrodomestico, TecnologiaHogar, Cliente, Vendedor, Venta, Garantia, GarantiaException`	Representan las entidades del negocio y sus reglas (por ejemplo, cómo se calcula la vigencia de una garantía).
Gestión	`PuntoTecno`	Guarda las listas de clientes, vendedores, productos y ventas. Ofrece las operaciones: altas, modificación, registro de ventas, búsquedas, filtros y ordenación.
Interfaz	`MenuPrincipal, VentanaRegistrarVenta, VentanaConsultaGarantia, VentanaListarVentas, VentanaAltaCliente, VentanaAltaProducto, VentanaModificarCliente, PanelFormulario, PanelTablaVentas, Formato`	Menú de selección y pantallas. PanelFormulario, PanelTablaVentas y Formato son piezas reutilizables para no repetir código.
Arranque	`PuntoTecnoApp`	Contiene el método main: crea el sistema con sus datos y abre el menú principal.

En total son 21 archivos, todos comentados por estructura (atributos, constructor, métodos y secciones), para facilitar su lectura.

Del diseño del TP2 al código

Las clases del diagrama de diseño del TP2 se llevaron a Java con pocos ajustes. El cambio más importante es el de Producto: en el TP2 tenía un atributo categoría de tipo texto; en el código la categoría pasó a ser un tipo de objeto (una subclase), de modo que cada categoría "sabe" cuántos días de garantía le corresponden. Esto reemplaza en memoria a la tabla categorias del modelo relacional.

Elemento del TP2	Clase en Java	Observación
Cliente	`Cliente`	Incluye actualizarDatos(nombre, contacto), tal como figuraba en el diseño.
Vendedor	`Vendedor`	Legajo y nombre.
Producto (con categoría)	`Producto + 3 subclases`	Clase abstracta; las subclases definen la categoría y el período de garantía.
Venta	`Venta`	Al crearse genera su Garantia (caso de uso incluido "Calcular garantía").
Garantía	`Garantia`	calcularVigencia(fecha) devuelve "Vigente" o "Vencida".
GestorVentas / GestorGarantias (análisis)	`PuntoTecno`	Clase gestora que coordina las operaciones. El cálculo de la garantía quedó dentro de Venta y Garantia, donde están los datos.
MenuPuntoTecno (interfaz)	`MenuPrincipal y Ventana*`	Menú principal y una pantalla por cada caso de uso.

Abstracción

La clase Producto es abstracta: representa "un producto del local" sin saber de qué categoría es, y no se puede instanciar directamente. Declara dos métodos abstractos que cada subclase está obligada a implementar. Del objeto solo se deja visible lo relevante para el problema (nombre, precio, categoría y días de garantía); lo demás queda contenido dentro de la clase.

Clase abstracta Producto (extracto)

```
public abstract class Producto {
    protected int idProducto;
    protected String nombre;
    protected double precio;

    public Producto(int idProducto, String nombre, double precio) {
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException(
                "El nombre del producto no puede estar vacío.");
        }
        if (precio < 0) {
            throw new IllegalArgumentException(
                "El precio no puede ser negativo.");
        }
        this.idProducto = idProducto;
        this.nombre = nombre.trim();
        this.precio = precio;
    }

    public abstract int getPeriodoGarantiaDias();   // cada categoría define el suyo
    public abstract String getCategoria();
}
```

Encapsulamiento
Los atributos de todas las clases son privados (private) o, en el caso de la clase abstracta Producto, protegidos (protected) para que solo las subclases puedan accederlos. Desde afuera se trabaja únicamente mediante métodos públicos. Además:

- Las validaciones (nombre vacío, precio negativo, contacto vacío) están dentro de las propias clases, por lo que no es posible crear un objeto con datos inválidos.
- PuntoTecno mantiene sus ArrayList privados y sus métodos getVentas(), getClientes(), etc. devuelven una copia de la lista, de manera que ninguna otra clase puede modificarla sin pasar por los métodos del sistema.
- Los métodos auxiliares (por ejemplo, la validación de Cliente) son privados: son un detalle interno que no le interesa al resto del sistema.

Getter que protege la lista interna

```
public List<Venta> getVentas() { return new ArrayList<>(ventas); }
```

Herencia
Las categorías de producto del negocio se modelaron como tres subclases de Producto: LineaBlanca (365 días), PequenoElectrodomestico (180 días) y TecnologiaHogar (90 días), que son los períodos definidos en el TP2. Cada una hereda los atributos y los getters de Producto, invoca al constructor de la clase padre con super(...) y sobrescribe los métodos abstractos con @Override.
Subclase LineaBlanca

```
public class LineaBlanca extends Producto {
    private static final int PERIODO_DIAS = 365;

    public LineaBlanca(int idProducto, String nombre, double precio) {
        super(idProducto, nombre, precio);
    }

    @Override
    public int getPeriodoGarantiaDias() { return PERIODO_DIAS; }

    @Override
    public String getCategoria() { return "Línea blanca"; }
}
```

Polimorfismo
El polimorfismo permite que el sistema trate a todos los productos de la misma manera sin importar su categoría. Cuando se crea una Venta, esta le pregunta al producto cuántos días de garantía tiene, sin saber si es de línea blanca, un pequeño electrodoméstico o tecnología del hogar: cada subclase responde su propio valor. Así se implementa el caso de uso incluido "Calcular garantía" en un único lugar y sin estructuras if/else por categoría.
Constructor de Venta: crea la garantía de forma automática (extracto)

```
public Venta(int numero, LocalDate fecha, Cliente cliente,
             Vendedor vendedor, Producto producto)
        throws GarantiaException {
    if (fecha == null || cliente == null
            || vendedor == null || producto == null) {
        throw new GarantiaException(
            "La venta necesita fecha, cliente, vendedor y producto.");
    }
    // ... se asignan los atributos ...

    // POLIMORFISMO: el producto responde su propio período de garantía
    LocalDate fin = fecha.plusDays(producto.getPeriodoGarantiaDias());
    this.garantia = new Garantia(fecha, fin);
}
```

También se aplica al dar de alta un producto: la pantalla crea la subclase que corresponde a la categoría elegida, pero la guarda como un `Producto` genérico.

VentanaAltaProducto: se crea la subclase y se guarda como Producto

```
Producto nuevo;
switch (comboCategoria.getSelectedIndex()) {
    case 0:  nuevo = new LineaBlanca(id, nombre, precio); break;
    case 1:  nuevo = new PequenoElectrodomestico(id, nombre, precio); break;
    default: nuevo = new TecnologiaHogar(id, nombre, precio); break;
}
sistema.agregarProducto(nuevo);
```

La clase `Garantia` resuelve el estado comparando la fecha de consulta con el vencimiento. El día exacto del vencimiento todavía se considera "Vigente", coherente con los casos de prueba CP02, CP05 y CP08 del TP2.

Garantia.calcularVigencia()

```
public String calcularVigencia(LocalDate fechaConsulta) {
    if (fechaConsulta.isAfter(fechaFin)) {
        return "Vencida";
    }
    return "Vigente";
}
```

Constructores

Siguiendo la indicación de la cátedra, no se utilizaron constructores por defecto. Todos los objetos se crean con un constructor adecuado a su propósito, que recibe los datos necesarios, los valida y los asigna; de esta forma un objeto nunca existe a medias.

Clase	Constructor	Qué hace
`Cliente`	Cliente(int, String, String)	Valida nombre y contacto antes de asignarlos.
`Vendedor`	Vendedor(int, String)	Valida el nombre.
`Producto` y subclases	LineaBlanca(int, String, double), etc.	Las subclases llaman a super(...); Producto valida nombre y precio.
`Garantia`	Garantia(LocalDate, LocalDate)	Verifica que el fin no sea anterior al inicio; lanza GarantiaException.
`Venta`	Venta(int, LocalDate, Cliente, Vendedor, Producto)	Valida los datos y crea la Garantia asociada.
`PuntoTecno`	PuntoTecno(String)	Inicializa las listas vacías y carga los datos de prueba.

Estructuras condicionales y repetitivas

Estructura	Dónde se utiliza
if / else	Cálculo de vigencia (Garantia), validaciones de datos, decisiones en la búsqueda de ventas por número, cliente o producto.
switch	Selección de la subclase de producto según la categoría elegida (VentanaAltaProducto).
for-each	Búsquedas lineales por id, generación del siguiente identificador, filtros por cliente, producto y rango de fechas, carga de las tablas y los combos.
while	Búsqueda binaria y ordenación por inserción (PuntoTecno).

Algoritmos de búsqueda y ordenación

Aunque la consigna los plantea como opcionales, se incorporaron porque resuelven necesidades reales del sistema. Se programaron a mano para que el algoritmo se vea en el código.

Búsqueda binaria de una venta por número (RFS06). Las ventas se agregan siempre con número creciente, por lo que la lista está ordenada por número. En cada paso se compara con el elemento del medio y se descarta una mitad de la lista.

PuntoTecno.buscarVenta()

```
public Venta buscarVenta(int numero) {
    int inicio = 0;
    int fin = ventas.size() - 1;
    while (inicio <= fin) {
        int medio = (inicio + fin) / 2;
        int numeroMedio = ventas.get(medio).getNumero();
        if (numeroMedio == numero) {
            return ventas.get(medio);
        } else if (numeroMedio < numero) {
            inicio = medio + 1;   // buscar en la mitad derecha
        } else {
            fin = medio - 1;      // buscar en la mitad izquierda
        }
    }
    return null;   // no existe
}
```

Ordenación por inserción de las ventas por fecha (RFS09). El algoritmo trabaja sobre una copia de la lista, para no alterar el orden por número que necesita la búsqueda binaria. Permite ordenar de la más reciente a la más antigua o al revés.

PuntoTecno.ordenarPorFecha() (extracto)

```
List<Venta> copia = new ArrayList<>(lista);
for (int i = 1; i < copia.size(); i++) {
    Venta actual = copia.get(i);
    int j = i - 1;
    // Desplaza a la derecha los elementos que deben ir después de "actual"
    while (j >= 0 && debeIrDespues(copia.get(j), actual, ascendente)) {
        copia.set(j + 1, copia.get(j));
        j--;
    }
    copia.set(j + 1, actual);
}
return copia;
```

Tratamiento y manejo de excepciones

Se creó una excepción propia, `GarantiaException` (equivalente a TurbinaExcepcion del material de la cátedra), que se lanza cuando no se puede crear una venta o una garantía. Además se tratan las excepciones propias de Java que pueden surgir al interactuar con la interfaz gráfica. En ningún caso el programa se cierra: se informa el problema en un mensaje claro y el usuario puede corregir el dato.

Excepción	Cuándo ocurre	Dónde se trata
`GarantiaException` (propia)	Venta sin fecha, cliente, vendedor o producto; garantía con fechas incoherentes.	VentanaRegistrarVenta
`DateTimeParseException`	Fecha mal escrita o inexistente (ej: 31/02/2026).	VentanaRegistrarVenta, VentanaListarVentas
`NumberFormatException`	Precio o número de venta que no son numéricos.	VentanaAltaProducto, VentanaConsultaGarantia

Excepción	Cuándo ocurre	Dónde se trata
`IllegalArgumentException`	Nombre o contacto vacíos, precio negativo, rango de fechas invertido, cliente inexistente.	Altas, modificación y listado

El material de la cátedra plantea el manejo de InputMismatchException, propia de la lectura por consola con Scanner. Como este prototipo usa interfaz gráfica, el mismo problema (ingresar un valor no numérico) se manifiesta como NumberFormatException, y se trata de la misma manera con try y catch.

GarantiaException

```
public class GarantiaException extends Exception {
    public GarantiaException(String mensaje) {
        super(mensaje);
    }
}
```

VentanaRegistrarVenta.registrar() (extracto)

```
try {
    LocalDate fecha = Formato.parseFecha(txtFecha.getText());
    // ^ puede lanzar DateTimeParseException
    Venta venta = sistema.registrarVenta(cliente, vendedor, producto, fecha);
    // ^ puede lanzar GarantiaException
    JOptionPane.showMessageDialog(this, "Venta registrada correctamente...");
    dispose();
} catch (DateTimeParseException ex) {
    JOptionPane.showMessageDialog(this,
        "La fecha es inválida. Use el formato DD/MM/AAAA.",
        "Fecha inválida", JOptionPane.WARNING_MESSAGE);
} catch (GarantiaException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
        "No se pudo registrar la venta", JOptionPane.ERROR_MESSAGE);
}
```

Menú de selección y pantallas

El menú principal es la ventana de arranque del sistema y desde ella se navega a todas las funciones. Cada opción abre una pantalla propia y, al cerrarla, se vuelve al menú. La opción Salir (y la X de la ventana) piden confirmación antes de cerrar el programa.

Opción del menú	Pantalla	Requerimiento
1. Registrar venta	`VentanaRegistrarVenta`	RFS02, RFS03
2. Dar de alta producto	`VentanaAltaProducto`	RFS04
3. Dar de alta cliente	`VentanaAltaCliente`	RFS05
4. Modificar cliente	`VentanaModificarCliente`	RFS10
5. Consultar estado de garantía	`VentanaConsultaGarantia`	RFS06, RFS07, RFS08
6. Listar ventas	`VentanaListarVentas`	RFS09 (y RFS07/RFS08 por la columna de estado)
7. Salir	`MenuPrincipal`	Cierre con confirmación

La pantalla de registrar venta muestra, en tiempo real, los días de garantía y la fecha de vencimiento que se aplicarán según el producto y la fecha elegidos, y sus botones "Nuevo cliente" y "Nuevo producto" cubren el flujo alternativo del caso de uso definido en el TP1 (si el cliente o el producto no existen, se pueden dar de alta sin salir de la pantalla).

Alcance de esta entrega y coherencia con el TP2

- **Alcance.** El prototipo implementa los requerimientos RFS02 a RFS10. El ingreso con usuario y contraseña (RFS01) no forma parte de esta primera versión operacional y queda para una etapa posterior.
- **Persistencia.** Los datos viven en memoria: al cerrar el programa se pierden las altas y ventas hechas durante la ejecución y se vuelve a cargar el conjunto inicial. La persistencia en MySQL con JDBC corresponde al TP4.
- **Datos de prueba.** Se cargaron los mismos registros del script puntotecno.sql, pero las garantías no se cargan a mano: las calcula cada venta según su categoría. Al hacerlo se detectaron dos diferencias en los datos que se habían cargado manualmente en el TP2: la venta 2 (Notebook, categoría tecnología del hogar, 90 días) figuraba con vencimiento 15/08/2027 cuando corresponde 13/11/2026, y la venta 4 (Microondas, 180 días) figuraba con vencimiento 30/07/2026 cuando corresponde 31/07/2026. Además, esa venta tenía el estado "Vigente" guardado, aunque ya había vencido. Esto confirma la conveniencia de calcular el estado en lugar de almacenarlo; los registros del script SQL se corregirán en el TP4 para que la base de datos y el programa coincidan.

Presentación del desarrollo en Java

Repositorio del proyecto

El código fuente completo del proyecto se encuentra publicado en GitHub:
https://github.com/Thiago-BF/SeminarioDePracticasDeInformatica-Siglo21/tree/TP3/PuntoTecno

Cómo compilar y ejecutar el programa

1. Abrir la carpeta del proyecto con IntelliJ IDEA (File > Open).
2. Configurar el SDK en File > Project Structure > Project (se utilizó JDK 22; sirve cualquier JDK 11 o superior).
3. Marcar la carpeta src como Sources Root (clic derecho sobre src > Mark Directory as > Sources Root).
4. Abrir la clase PuntoTecnoApp y ejecutar su método main con el botón Run.

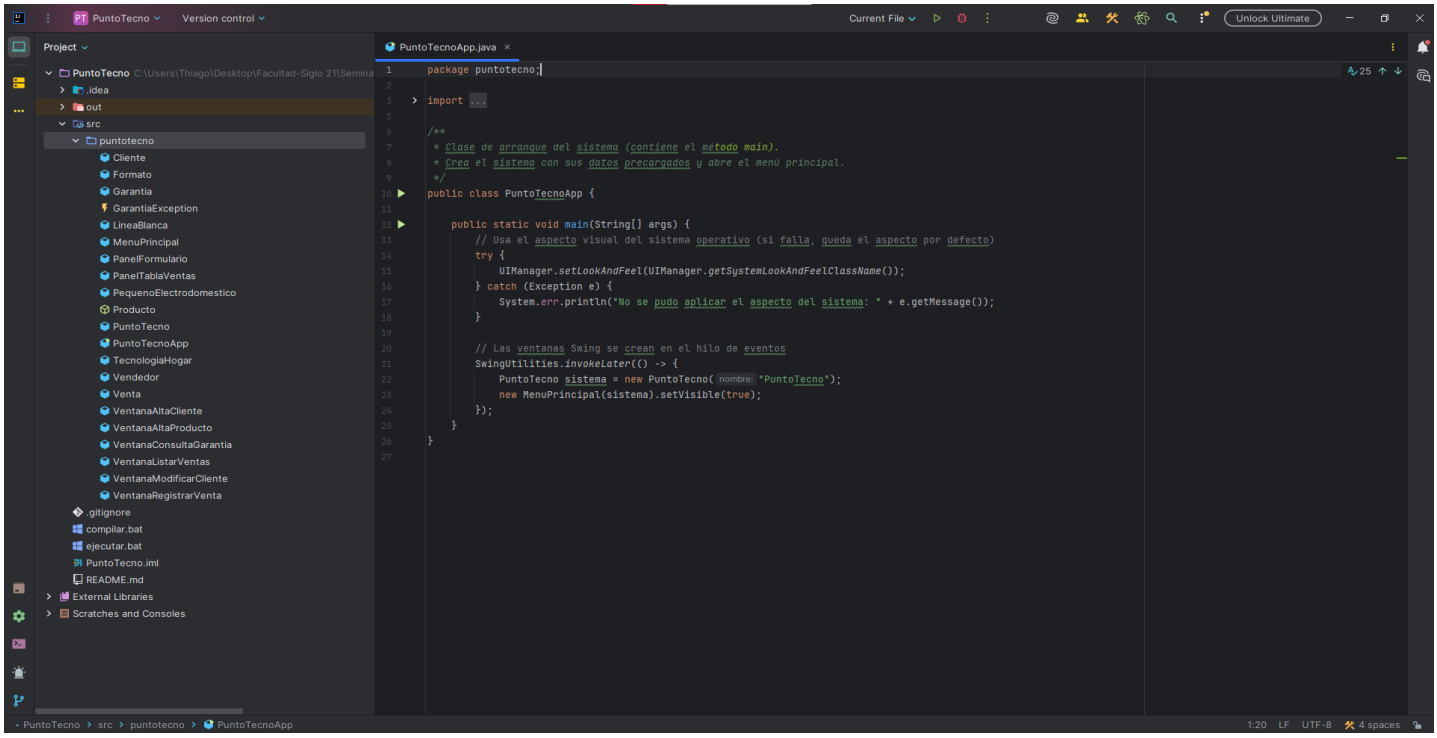
También puede compilarse y ejecutarse desde la consola, ubicándose en la carpeta del proyecto:

```
javac -encoding UTF-8 -d bin src/puntotecno/*.java
java -cp bin puntotecno.PuntoTecnoApp
```

Evidencia de funcionamiento

Las siguientes capturas corresponden a una única ejecución del programa realizada el 15/10/2026, a partir de los datos de prueba precargados. El estado de cada garantía (Vigente o Vencida) se calcula con la fecha del sistema del momento de la ejecución.

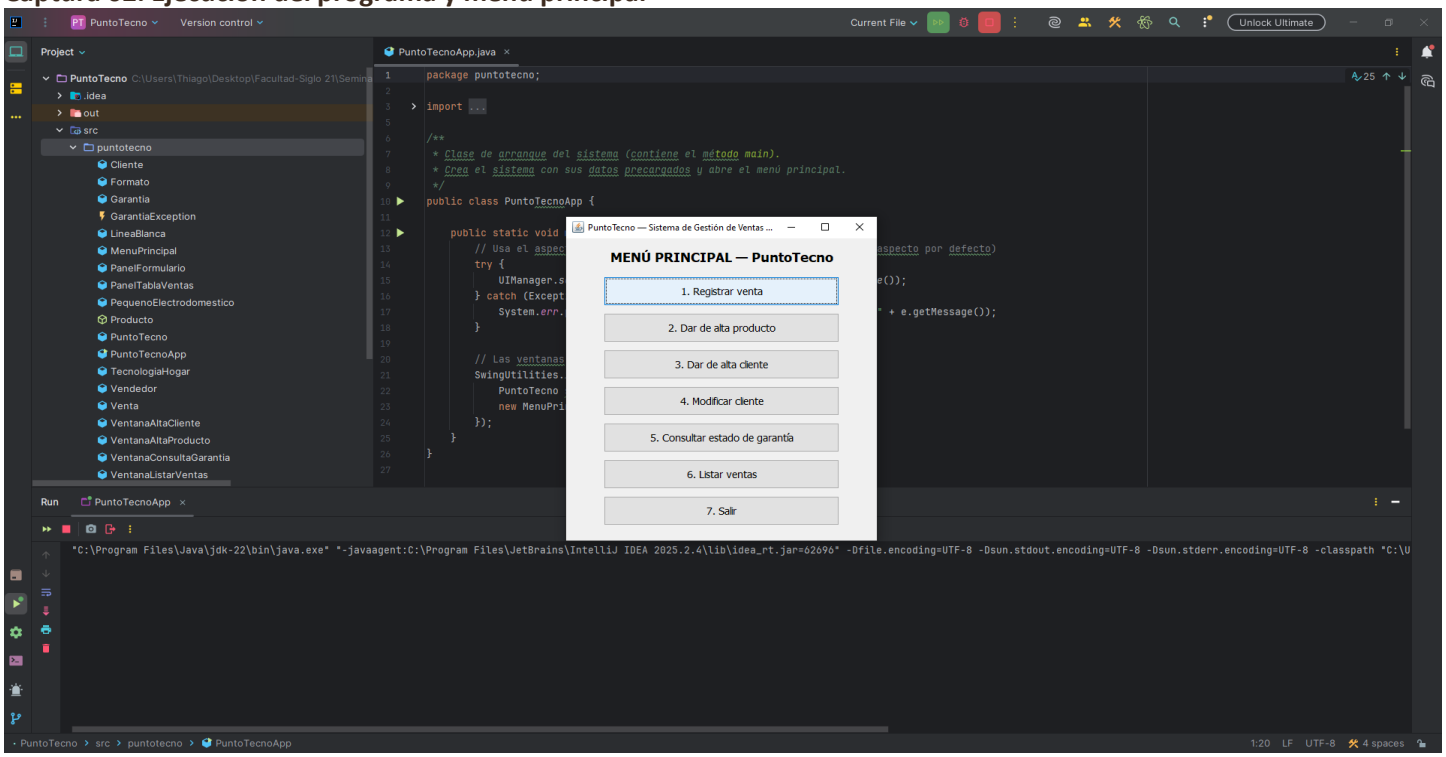
Captura 01. Estructura del proyecto en IntelliJ IDEA



Fuente: elaboración propia

Se observa el paquete puntotecno con las 21 clases del sistema, organizadas en modelo, gestión, interfaz y arranque. Evidencia la estructura del proyecto y el uso de una clase por responsabilidad.

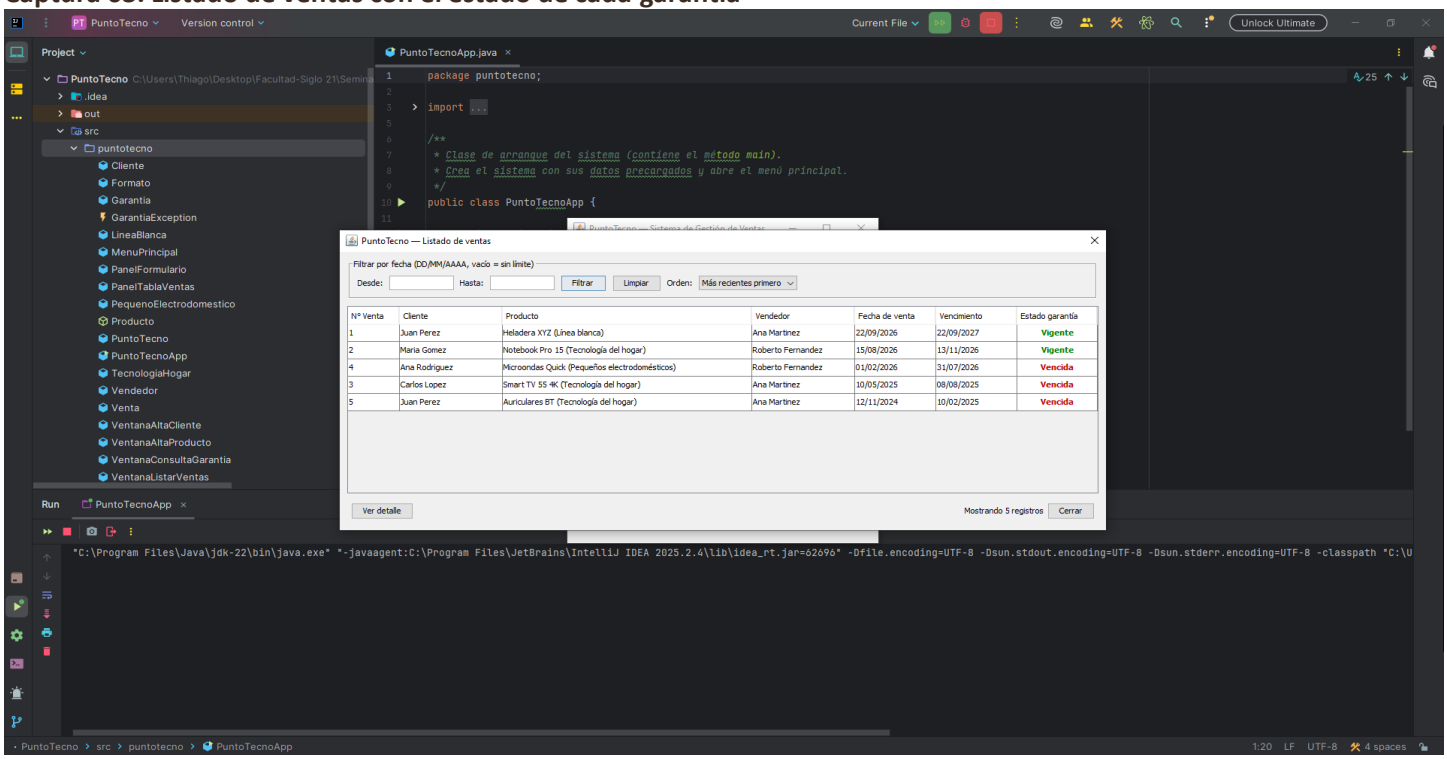
Captura 02. Ejecución del programa y menú principal



Fuente: elaboración propia

El programa compila y se ejecuta correctamente: el panel Run de IntelliJ no muestra errores y se abre el menú principal con las siete opciones. Cumple con el requisito de disponer de un menú de selección.

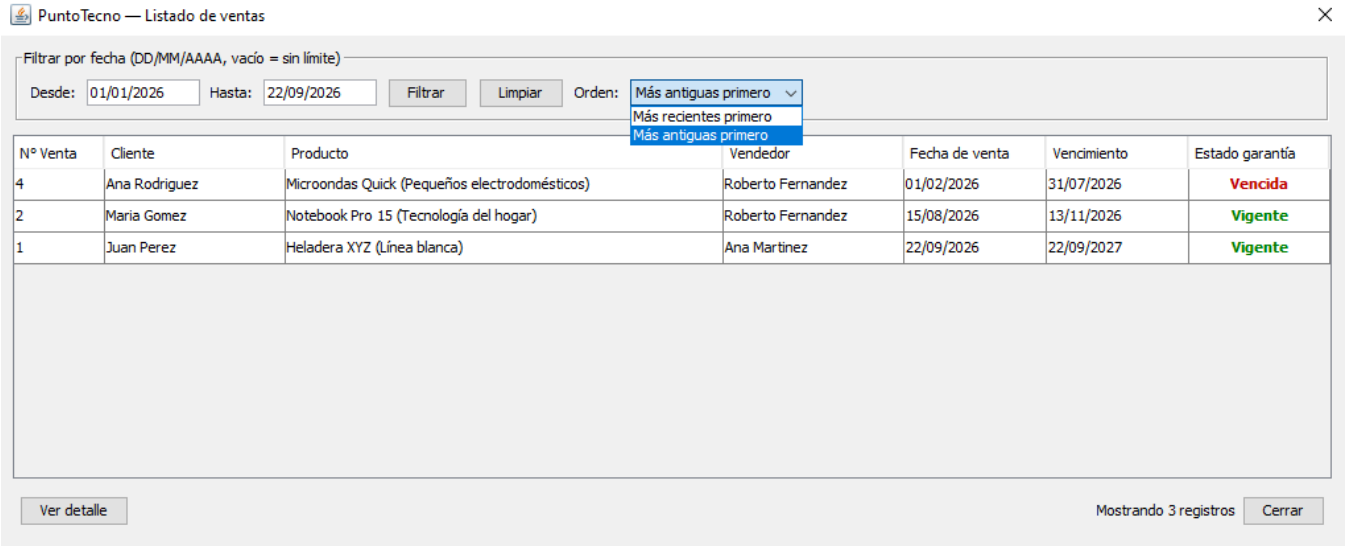
Captura 03. Listado de ventas con el estado de cada garantía



Fuente: elaboración propia

Las cinco ventas precargadas ordenadas de la más reciente a la más antigua. La columna "Estado garantía" no se lee de un dato guardado, sino que se calcula: las ventas 1 y 2 están vigentes (verde) y las ventas 3, 4 y 5 están vencidas (rojo).

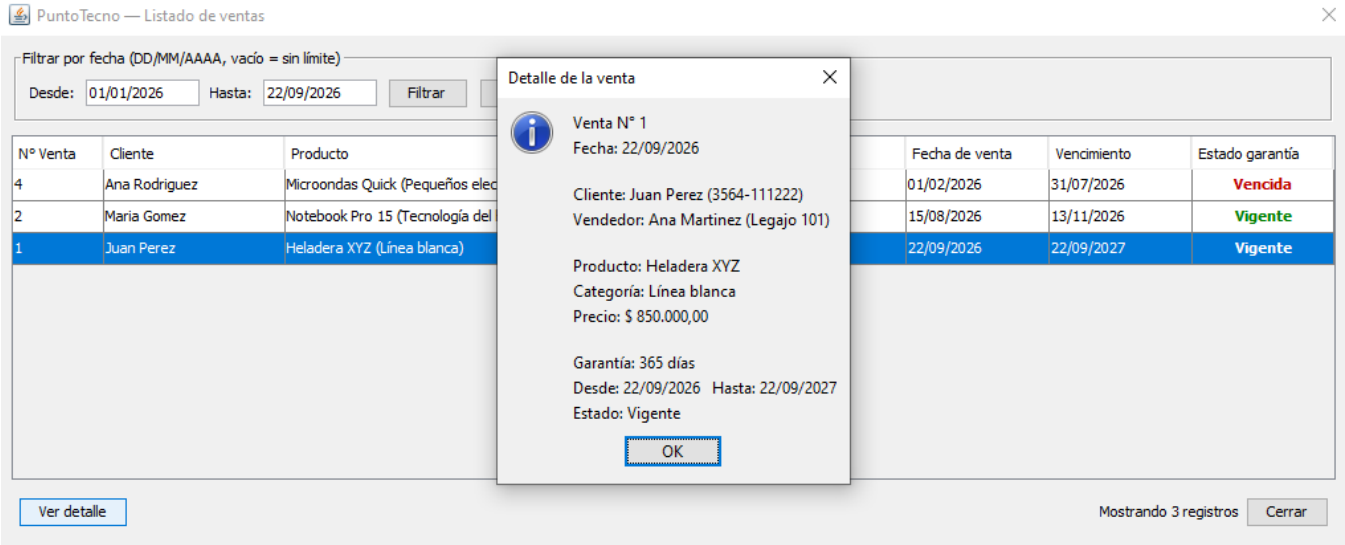
Captura 04. Filtro por fechas y ordenación



Fuente: elaboración propia

Con el orden "Más antiguas primero" y el rango de fechas del 01/01/2026 al 22/09/2026 se listan tres ventas (4, 2 y 1). Muestra el uso de estructuras condicionales y repetitivas para filtrar, y del algoritmo de ordenación por inserción.

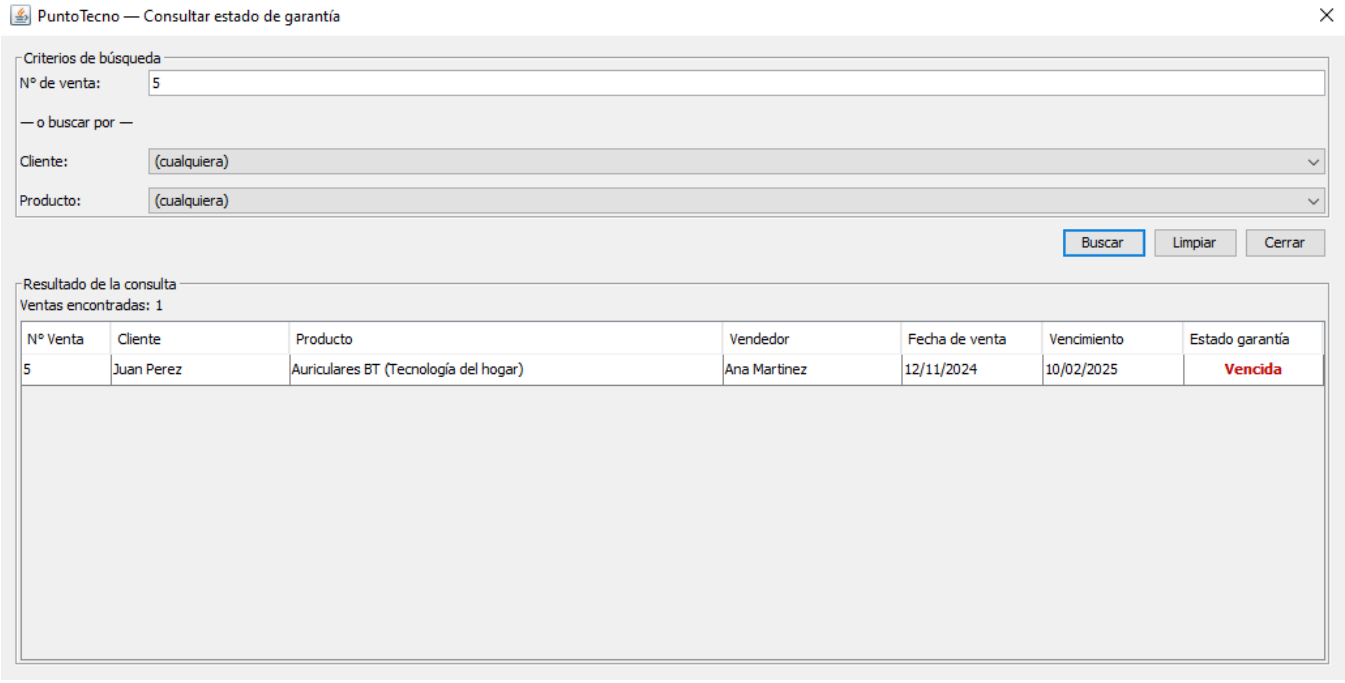
Captura 05. Detalle de una venta



Fuente: elaboración propia

Al seleccionar la venta 1 y presionar "Ver detalle" se muestran los datos del cliente, el vendedor, el producto y la garantía. Evidencia que los objetos Venta, Cliente, Vendedor, Producto y Garantia están correctamente relacionados.

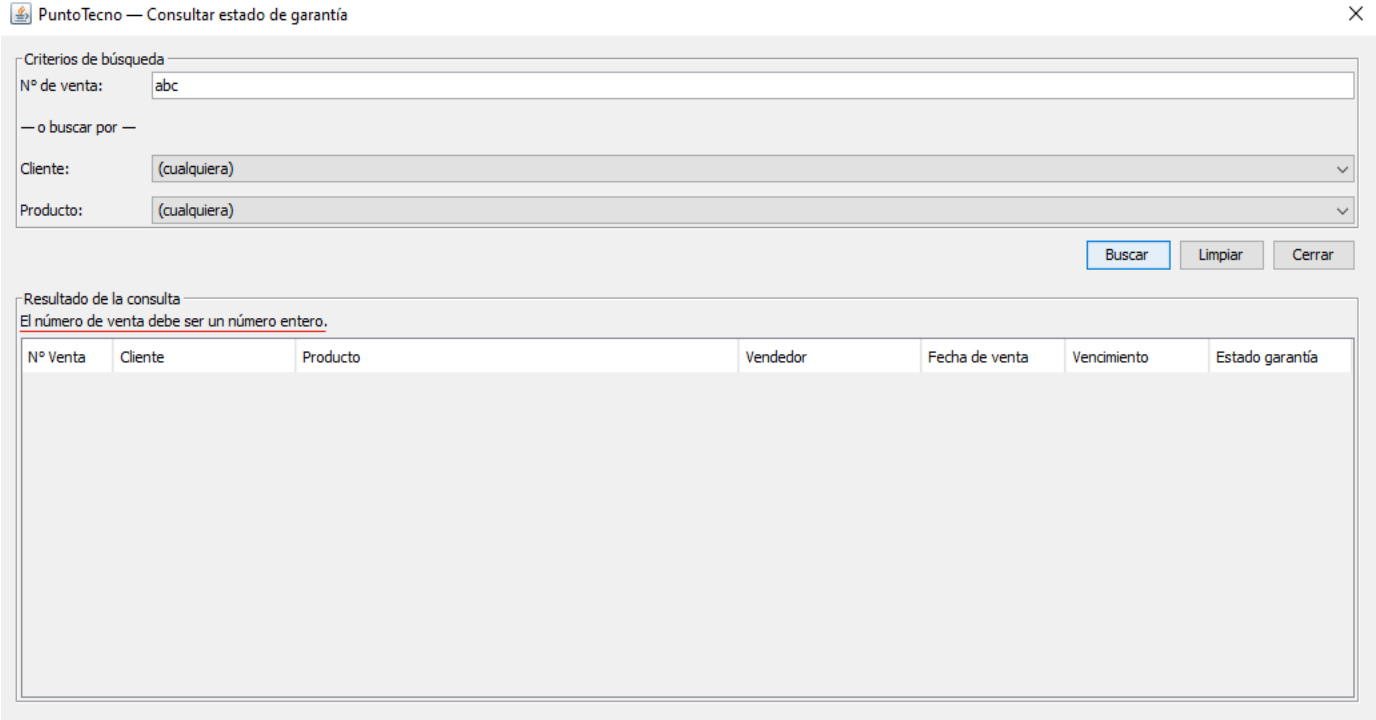
Captura 06. Consulta de garantía por número de venta



Fuente: elaboración propia

La búsqueda del número de venta 3 (algoritmo de búsqueda binaria) devuelve el Smart TV de Carlos Lopez, vendido el 10/05/2025. Al ser tecnología del hogar (90 días), venció el 08/08/2025 y el sistema informa "Vencida".

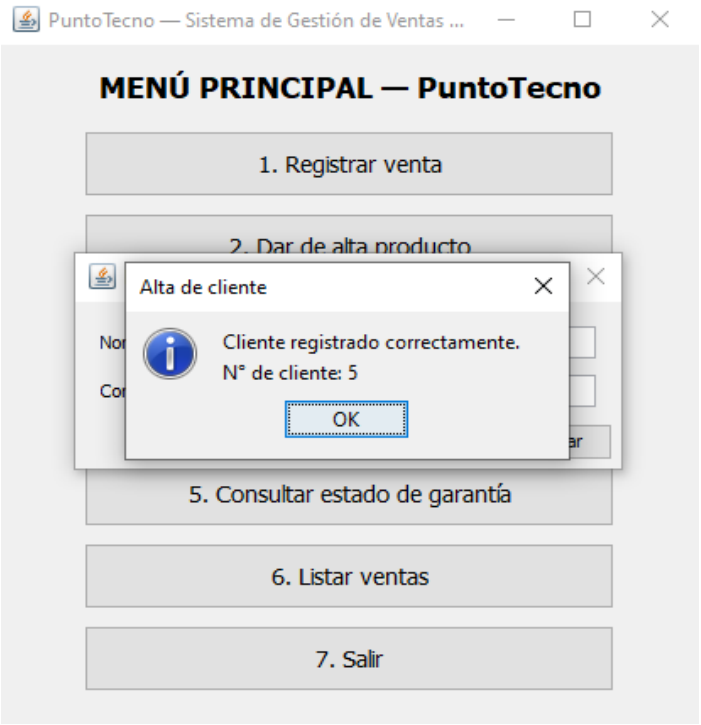
Captura 07. Excepción por dato no numérico



Fuente: elaboración propia

Al ingresar "abc" como número de venta se captura la excepción NumberFormatException con try y catch y se informa al usuario que debe ingresar un número entero. El programa no se interrumpe.

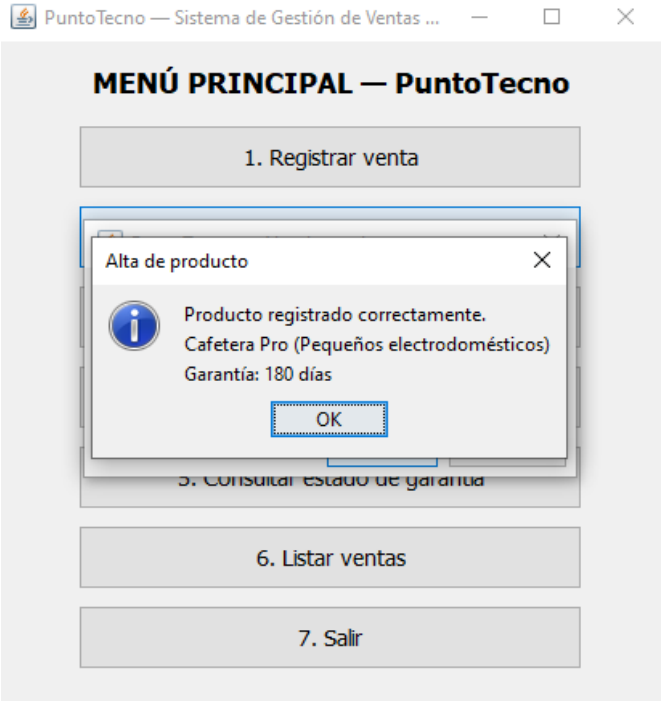
Captura 08. Alta de cliente



Fuente: elaboración propia

Se da de alta a Laura Gimenez. El sistema crea el objeto Cliente mediante su constructor con parámetros y le asigna automáticamente el número de cliente 5, siguiente al último existente.

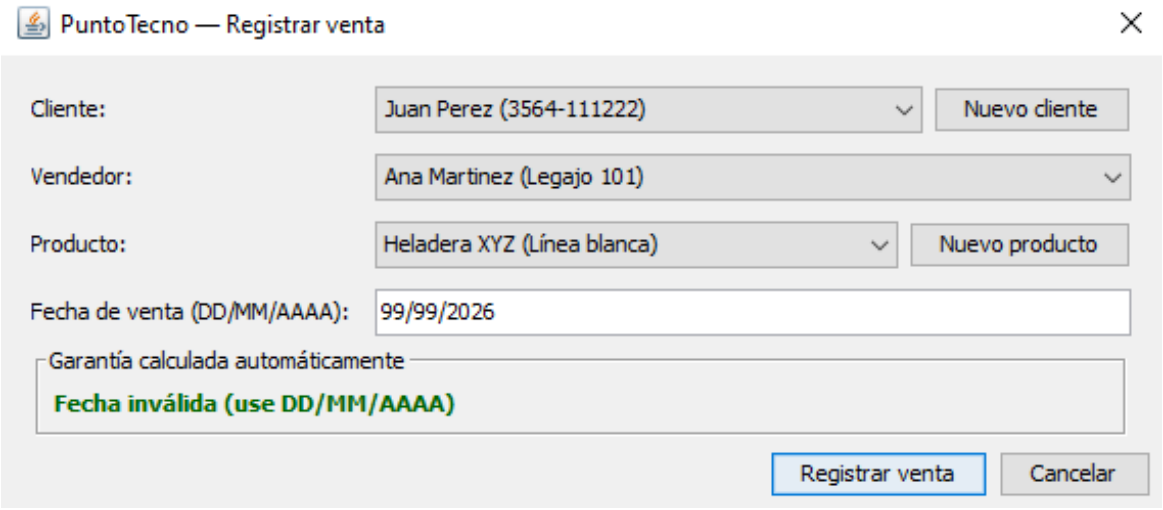
Captura 09. Alta de producto



Fuente: elaboración propia

Se da de alta la "Cafetera Pro" en la categoría pequeños electrodomésticos. Se instancia la subclase PequenoElectrodomestico y el sistema informa su garantía de 180 días: es un ejemplo de herencia y polimorfismo.

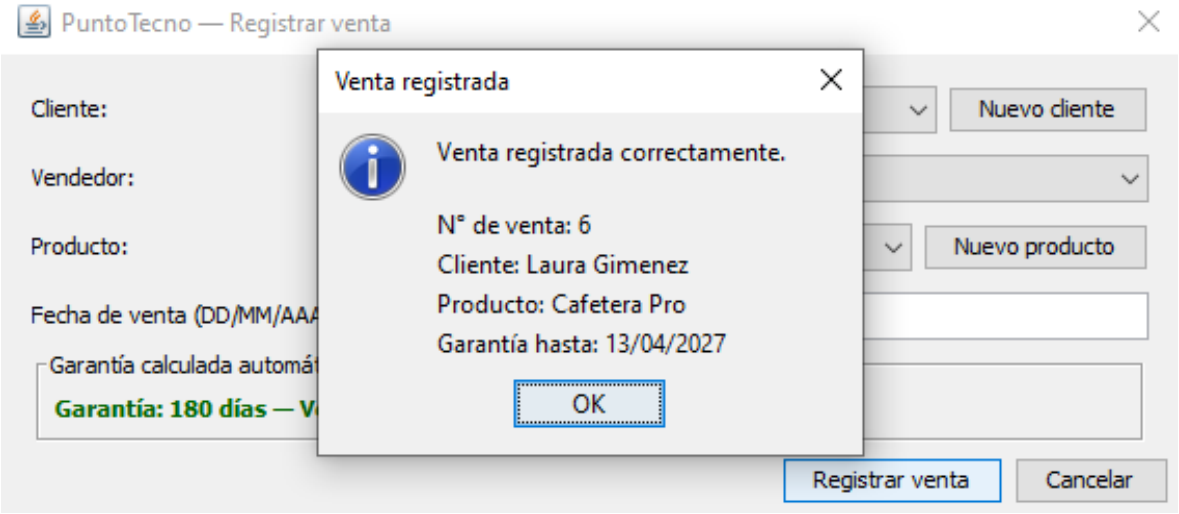
Captura 10. Excepción por fecha inválida



Fuente: elaboración propia

Al escribir una fecha inexistente (99/99/2026) en "Registrar venta", la excepción DateTimeParseException es tratada y la pantalla avisa "Fecha inválida" en el panel de garantía, sin cerrarse ni perder los demás datos cargados.

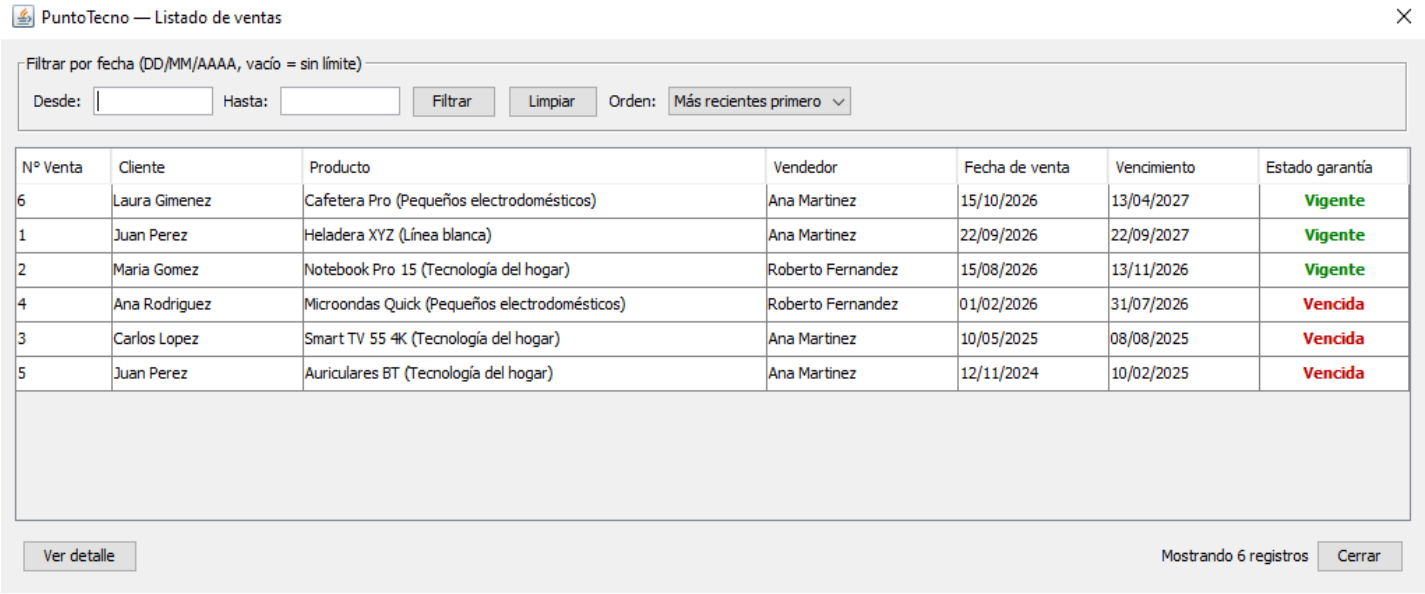
Captura 11. Registro de una venta con garantía automática



Fuente: elaboración propia

Se registra la venta de la Cafetera Pro a Laura Gimenez el 15/10/2026. El sistema le asigna el número de venta 6 y calcula la garantía hasta el 13/04/2027 (15/10/2026 más 180 días), sin ningún cálculo manual del vendedor.

Captura 12. Listado final con la venta registrada



N° Venta	Cliente	Producto	Vendedor	Fecha de venta	Vencimiento	Estado garantía
6	Laura Gimenez	Cafetera Pro (Pequeños electrodomésticos)	Ana Martinez	15/10/2026	13/04/2027	Vigente
1	Juan Perez	Heladera XYZ (Línea blanca)	Ana Martinez	22/09/2026	22/09/2027	Vigente
2	Maria Gomez	Notebook Pro 15 (Tecnología del hogar)	Roberto Fernandez	15/08/2026	13/11/2026	Vigente
4	Ana Rodriguez	Microondas Quick (Pequeños electrodomésticos)	Roberto Fernandez	01/02/2026	31/07/2026	Vencida
3	Carlos Lopez	Smart TV 55 4K (Tecnología del hogar)	Ana Martinez	10/05/2025	08/08/2025	Vencida
5	Juan Perez	Auriculares BT (Tecnología del hogar)	Ana Martinez	12/11/2024	10/02/2025	Vencida

Fuente: elaboración propia

El listado muestra ahora seis ventas y la nueva aparece primera por ser la más reciente. Confirma que la venta y su garantía quedaron almacenadas en el ArrayList del sistema y disponibles para su consulta.

TP 4

Presentación del desarrollo en Java

Este trabajo práctico completa el prototipo de PuntoTecno iniciado en el TP3. En el TP3 el sistema ya contaba con una interfaz gráfica (Swing), clases del dominio, una clase abstracta, excepciones propias y colecciones ArrayList, pero los datos vivían solo en memoria: al cerrar el programa se perdían las altas y las ventas. En el TP4 se incorpora la **persistencia en MySQL mediante JDBC**, la **selección y aplicación del patrón de diseño MVC**, el **tratamiento de excepciones propias de la base de datos**, el uso de una **interfaz** junto a la clase abstracta existente y el uso complementario de **arreglos y ArrayList**.

El sistema conserva el alcance definido en el TP1: registrar ventas con cálculo automático de garantía, dar de alta productos y clientes, modificar clientes, consultar el estado de una garantía y listar ventas. Lo que cambia es de dónde vienen y hacia dónde van los datos: ahora se leen de la base puntotecno al iniciar el programa y cada alta, modificación o venta se guarda en ella.

Observaciones de entregas anteriores

Al integrar MySQL se revisó la coherencia entre la documentación, la base de datos y el código, y se corrigieron los puntos que se detallan a continuación. Dos de ellos ya habían sido anunciados en el TP3 como tareas para esta etapa.

Mejoras incorporadas respecto de entregas anteriores

Punto detectado	Situación anterior	Corrección en el TP4
Vencimientos de las ventas 2 y 4	En el script del TP2 figuraban con 15/08/2027 y 30/07/2026; las categorías (90 y 180 días) indican 13/11/2026 y 31/07/2026. (Anunciado en el TP3.)	Se corrigieron en `sql/puntotecno_tp4.sql`. Base y programa coinciden.
Estado de garantía guardado	La venta 4 tenía “Vigente” guardado aunque ya había vencido. El TP3 ya decidió calcular el estado y no confiar en el dato guardado.	Al iniciar, el programa corrige con un UPDATE la columna estado de las garantías según la fecha actual.
Script SQL del TP2 no reutilizable	Sin IF NOT EXISTS y con UPDATE/INSERT/DELETE de demostración al final: importarlo completo alteraba los datos.	Se separó en puntotecno_tp4.sql (estructura y datos, reejecutable) y `consultas_demostracion.sql` (verificaciones). El original se conserva.
Relación 1 a 1 venta-garantía	El TP1 la define así, pero la tabla garantias no lo garantizaba.	Se agregó UNIQUE sobre `garantias.numero_venta`.

Punto detectado	Situación anterior	Corrección en el TP4
Arquitectura descripta en el TP2	El diagrama de despliegue muestra una aplicación web con navegador, Apache y XAMPP; el prototipo implementado desde el TP3 es de escritorio (Swing).	Se aclara aquí: el prototipo es de escritorio y se conecta a MySQL por JDBC, que es el enlace que ya figuraba en el diagrama del TP2. XAMPP se utiliza como proveedor del servidor MySQL.

Patrón de diseño: Modelo-Vista-Controlador

Qué problema resuelve

Una interfaz gráfica que además accede a una base de datos tiende a mezclar en las mismas clases la presentación, las reglas del negocio y el SQL. Eso vuelve difícil cambiar una parte sin romper otra. El patrón MVC separa el sistema en tres responsabilidades: el **Modelo** (datos, reglas de negocio y acceso a la base), la **Vista** (lo que ve y toca el usuario) y el **Controlador** (recibe las acciones de la vista y las coordina con el modelo) (Debrauwer, 2013).

Por qué es adecuado para PuntoTecno

- El material de la materia lo propone como patrón para este tipo de prototipo, y el RNF06 del TP1 ya establecía que la persistencia y consulta de datos se implementaría con MVC.
- El proyecto ya tenía una separación natural entre clases del dominio, una clase gestora (PuntoTecno) y pantallas Swing. Faltaba la pieza que las comunica y el acceso a MySQL, que son justamente el controlador y el modelo de datos.
- Tiene ventajas concretas para este sistema: si cambia una pantalla solo se toca la vista; si cambia una tabla, solo ModeloMySQL; las reglas de la garantía siguen en Venta y Garantia.

Alcance de la aplicación del patrón

Siguiendo el ejemplo del material (donde MVC se aplica a un paquete de conexión a la base), en PuntoTecno el patrón se aplicó de forma **progresiva sobre todas las operaciones que usan MySQL**: lectura inicial, alta de cliente, modificación de cliente, alta de producto y registro de venta. Las pantallas de consulta y listado acceden a los datos a través del mismo controlador. No se reescribieron las clases del dominio ni la interfaz del TP3.

Responsabilidades MVC en las clases reales

Rol	Clases	Responsabilidad
Modelo	Cliente, Vendedor, Producto (y subclases), Venta, Garantia, GarantiaException	Representan las entidades del negocio y sus reglas (por ejemplo, el cálculo de vigencia).
Modelo	PuntoTecno	Mantiene en memoria, en ArrayList, los datos cargados desde MySQL y ofrece búsquedas, filtros y ordenación.
Modelo (acceso a datos)	RepositorioPuntoTecno (interfaz), ModeloMySQL, ConexionBD, RegistroVenta	Único lugar que conoce JDBC y SQL: abre conexiones, ejecuta consultas y escrituras, y convierte los resultados en objetos.

Rol	Clases	Responsabilidad
Vista	MenuPrincipal, VentanaRegistrarVenta, VentanaAltaCliente, VentanaAltaProducto, VentanaModificarCliente, VentanaConsultaGarantia, VentanaListarVentas, PanelFormulario, PanelTablaVentas, Formato, Mensajes	Muestran datos y capturan las acciones del usuario. No contienen SQL.
Controlador	ControladorPuntoTecno	Recibe las acciones de la vista, valida, ordena las operaciones sobre MySQL y sobre la memoria, y devuelve resultados o errores.

Cómo se comunican

El flujo de una operación de escritura, tomando como ejemplo **Registrar venta**, es el siguiente:

- La vista `VentanaRegistrarVenta` toma cliente, vendedor, producto y fecha, y llama a `controlador.registrarVenta(...)`.
- El controlador pide al repositorio el siguiente número de venta y crea el objeto `Venta`. Al crearse, la venta calcula su garantía (caso de uso incluido "Calcular garantía").
- El controlador ordena `repositorio.insertarVenta(venta)`, que guarda la venta y la garantía en MySQL dentro de una transacción.
- Solo si MySQL confirmó el guardado, el controlador agrega la venta a las listas en memoria y devuelve el resultado a la vista, que muestra el mensaje de confirmación.
- Si algo falla, el modelo lanza `PersistenciaException`, el controlador no toca la memoria y la vista muestra el error.

El orden **validar, guardar en MySQL y recién después actualizar la memoria** evita que las listas queden con datos que no existen en la base.

Armado de las piezas al iniciar

La clase PuntoTecnoApp arma la cadena de objetos y abre el menú. Si no hay conexión, informa el motivo y no inicia, porque no tendría sentido operar con datos que no podrían guardarse.

```
ConexionBD conexion = ConexionBD.desdeArchivo(ConexionBD.ARCHIVO_POR_DEFECTO);
PuntoTecno sistema = new PuntoTecno("PuntoTecno");
ControladorPuntoTecno controlador =
        new ControladorPuntoTecno(sistema, new ModeloMySQL(conexion));

List<String> advertencias = controlador.iniciar();   // lee MySQL
new MenuPrincipal(controlador).setVisible(true);     // vista + controlador
```

Fuente: elaboración propia (PuntoTecnoApp.java, extracto).

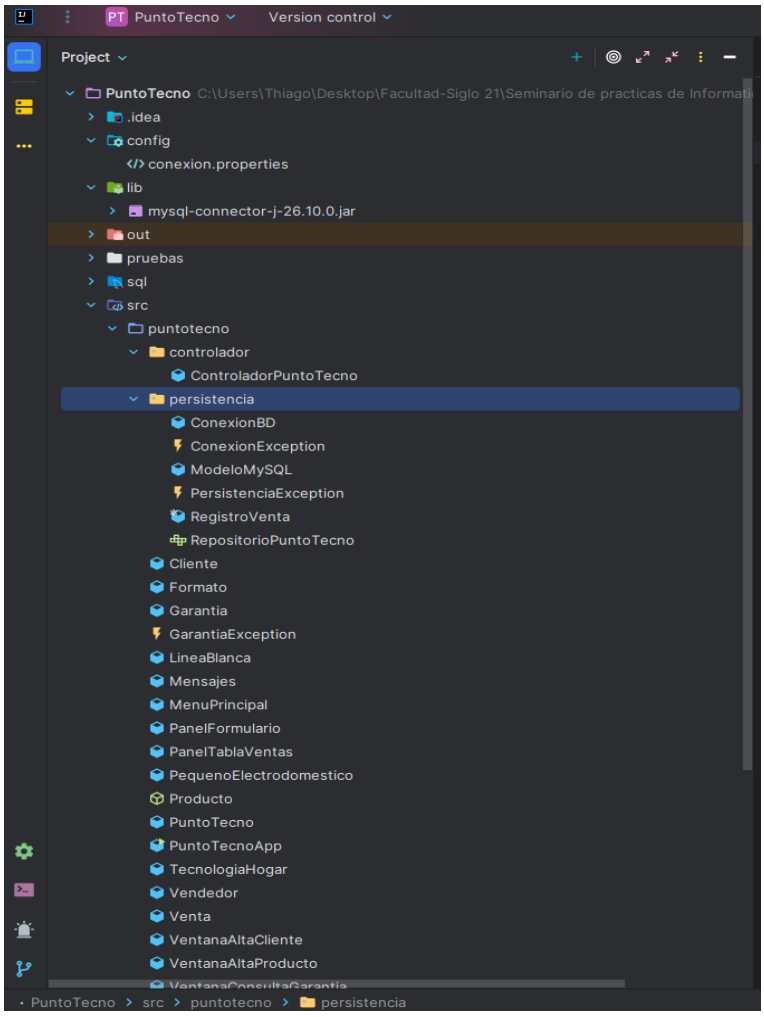
Organización de paquetes y clases

El proyecto pasó de 21 a 29 archivos de código en src. Las clases del TP3 se mantienen en el paquete puntotecno; las nuevas se ubicaron en dos paquetes según su rol.

Organización del código fuente

Paquete	Clases	Novedad en el TP4
puntotecno	Dominio (Producto, LineaBlanca, PequenoElectrodomestico, TecnologiaHogar, Cliente, Vendedor, Venta, Garantia, GarantiaException), gestora (PuntoTecno), vistas (MenuPrincipal y Ventana*, PanelFormulario, PanelTablaVentas, Formato), Mensajes, PuntoTecnoApp	Se agregó Mensajes. Se modificaron Producto y sus subclases, PuntoTecno, PuntoTecnoApp, MenuPrincipal y las seis ventanas.
puntotecno.controlador	ControladorPuntoTecno	Nuevo.
puntotecno.persistencia	ConexionBD, ConexionException, PersistenciaException, RepositorioPuntoTecno (interfaz), ModeloMySQL, RegistroVenta	Nuevo.

Además se incorporaron fuera de src: sql/puntotecno_tp4.sql y sql/consultas_demostracion.sql, la plantilla config/conexion.properties.example, la carpeta lib para el driver JDBC, los scripts compilar.bat y ejecutar.bat, el README.md y una prueba de integración por consola en pruebas/PruebaIntegracion.java.



Fuente: elaboración propia.

Acceso y persistencia de datos en MySQL

Preparación de la base de datos

La base se crea importando sql/puntotecno_tp4.sql (MySQL Workbench: File > Open SQL Script y ejecutar todo). El script elimina y vuelve a crear la base puntotecno con las seis tablas del modelo del TP2 (categorias, productos, clientes, vendedores, ventas y garantias) y carga los mismos datos de prueba que usa el prototipo, ya corregidos.



Fuente: elaboración propia.

Configuración de la conexión

Los datos de conexión no están escritos en el código fuente. Se leen del archivo `config/conexion.properties`, que cada persona crea en su equipo a partir de la plantilla `conexion.properties.example` y que está excluido del repositorio mediante `.gitignore`. De esta manera la contraseña no se publica en GitHub. El driver JDBC de MySQL (Connector/J) se coloca como archivo `.jar` en la carpeta `lib`.

```
# config/conexion.properties.example
db.url=jdbc:mysql://localhost:3306/puntotecno
db.usuario=root
db.clave=
```

Fuente: elaboración propia.

La clase `ConexionBD` carga ese archivo y abre una conexión nueva cada vez que se la solicita. Quien la recibe debe cerrarla, por lo que en el modelo siempre se utiliza `try-with-resources`.

```
public Connection obtener() throws ConexionException {
    try {
        return DriverManager.getConnection(url, usuario, clave);
    } catch (SQLException e) {
        throw new ConexionException(mensajeAmigable(e), e);
    }
}
```

Fuente: elaboración propia (ConexionBD.java, extracto).

Consultas: lectura de datos y presentación en la interfaz

`ModeloMySQL` implementa la interfaz `RepositorioPuntoTecno`. Todos sus métodos usan `PreparedStatement`, abren la conexión, la sentencia y el `ResultSet` en un `try-with-resources` y devuelven los resultados en `ArrayList`. Como ejemplo, la lectura de clientes:

```
public List<Cliente> listarClientes() throws PersistenciaException {
    String sql = "SELECT id_cliente, nombre, contacto FROM clientes ORDER BY id_cliente";
    List<Cliente> clientes = new ArrayList<>();
    try (Connection con = conexion.obtener();
         PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            clientes.add(new Cliente(rs.getInt("id_cliente"),
                    rs.getString("nombre"), rs.getString("contacto")));
        }
    } catch (SQLException e) {
        throw error("No se pudieron leer los clientes.", e);
    }
    return clientes;
}
```

Fuente: elaboración propia (ModeloMySQL.java, extracto).

Al leer los productos, el modelo usa el `id_categoria` para crear la subclase que corresponde (`LineaBlanca`, `PequenoElectrodomestico` o `TecnologiaHogar`). Las ventas se leen como `RegistroVenta` (solo identificadores) y el controlador las convierte en objetos `Venta` usando las mismas instancias de cliente, vendedor y producto de las listas, de modo que todo el sistema comparta los mismos objetos. La garantía de cada venta se **calcula** a partir de la fecha y de la categoría, igual que en el TP3, sin depender del vencimiento guardado.

Los resultados se muestran en las pantallas existentes: los combos de clientes, vendedores y productos, el listado de ventas con el estado de cada garantía y la consulta de garantía. Todas se alimentan de las listas cargadas desde MySQL.

Operaciones sobre MySQL

Operación	Sentencia	Qué hace	Desde la interfaz
Lectura inicial	SELECT sobre clientes, vendedores, productos, ventas y categorias	Carga los datos en las listas ArrayList	Al iniciar el programa
Alta de cliente	INSERT INTO clientes	Guarda el nuevo cliente con el siguiente id libre	3. Dar de alta cliente
Modificar cliente	UPDATE clientes SET nombre, contacto WHERE id_cliente	Actualiza los datos de un cliente. Si no existe, informa el error	4. Modificar cliente
Alta de producto	INSERT INTO productos	Guarda el producto con el id de su categoría	2. Dar de alta producto
Registrar venta	INSERT INTO ventas + INSERT INTO garantias (transacción)	Guarda la venta y su garantía juntas, o ninguna de las dos	1. Registrar venta
Sincronizar estados	UPDATE garantias SET estado = CASE WHEN fecha_fin >= hoy …	Corrige el estado guardado de las garantías según la fecha actual	Al iniciar el programa
Siguiente identificador	SELECT COALESCE(MAX(id), 0) + 1	Calcula el próximo id según lo que hay en MySQL	Altas y ventas

Modificaciones: transacción al registrar una venta

Registrar una venta implica dos escrituras relacionadas: la fila de `ventas` y la de `garantias`. Para que nunca quede una venta sin garantía se utiliza una **transacción**: se desactiva el auto-commit, se ejecutan los dos INSERT y recién entonces se confirma con `commit`. Si ocurre un error se deshace todo con `rollback`.

```
try (Connection con = conexion.obtener()) {
    try {
        con.setAutoCommit(false);
        // ... INSERT INTO ventas (...) VALUES (?, ?, ?, ?, ?)
        // ... INSERT INTO garantias (...) VALUES (?, ?, ?, ?, ?)
        con.commit();
    } catch (SQLException e) {
        try { con.rollback(); } catch (SQLException ex) { e.addSuppressed(ex); }
        throw error("No se pudo registrar la venta. No se guardó ningún dato.", e);
    }
}
```

Fuente: elaboración propia (ModeloMySQL.insertarVenta, extracto).

Modificaciones: sincronización del estado de las garantías

El TP3 decidió calcular el estado de la garantía en lugar de guardarlo, para que nunca quede desactualizado. La tabla `garantias` conserva igualmente la columna estado del modelo del TP2, por lo que al iniciar el programa se ejecuta un UPDATE que la deja coherente con la fecha del día. La sentencia solo modifica las filas cuyo estado no coincide, y el programa informa en la consola cuántas corrigió.

```
UPDATE garantias
SET estado = CASE WHEN fecha_fin >= ? THEN 'Vigente' ELSE 'Vencida' END
WHERE estado IS NULL
   OR estado <> CASE WHEN fecha_fin >= ? THEN 'Vigente' ELSE 'Vencida' END
```

Fuente: elaboración propia (ModeloMySQL.sincronizarEstadosGarantias, los parámetros son la fecha de hoy).

El criterio es el mismo que en `Garantia.calcularVigencia()`: el día exacto del vencimiento la garantía todavía está vigente.

Tratamiento y manejo de excepciones

Se conservó `GarantiaException` y el tratamiento de `NumberFormatException`, `DateTimeParseException` e `IllegalArgumentException` del TP3, y se agregaron dos excepciones propias para la interacción con MySQL. La clase principal es `PersistenciaException`; `ConexionException` es una subclase para los problemas al establecer la conexión, lo que permite a la interfaz distinguir "no hay conexión" de "falló una operación".

```
public class PersistenciaException extends Exception {
    public PersistenciaException(String mensajeParaElUsuario, Throwable causa) {
        super(mensajeParaElUsuario, causa);
    }
    // getDetalleTecnico(): tipo, mensaje, SQLState y código de error de la causa
}
```

Fuente: elaboración propia (PersistenciaException.java, extracto).

La excepción separa dos cosas: el **mensaje** que se muestra al usuario, escrito en lenguaje comprensible, y la **causa** (la `SQLException` original), que se conserva y se imprime en la consola con su SQLState y código de error para diagnosticar el problema. No hay bloques catch vacíos. La clase Mensajes de la vista muestra el error y las pantallas permanecen abiertas para que el usuario pueda reintentar.

Excepciones relacionadas con la base de datos

Situación	Excepción	Dónde se origina	Qué ve el usuario
Falta el archivo de configuración	ConexionException	ConexionBD.desdeArchivo	Indica copiar la plantilla como config/conexion.properties
MySQL apagado o url incorrecta	ConexionException	ConexionBD.obtener	"No se pudo conectar con el servidor MySQL…"
Usuario o contraseña incorrectos	ConexionException	ConexionBD.obtener	"MySQL rechazó el usuario o la contraseña"
Falta el driver .jar	ConexionException	ConexionBD.obtener	"No se encontró el driver JDBC de MySQL"
Clave duplicada o referencia inexistente	PersistenciaException (SQLState 23xxx)	ModeloMySQL (altas y ventas)	"Ya existe un registro con ese identificador…"

Situación	Excepción	Dónde se origina	Qué ve el usuario
Falla en una consulta o escritura	PersistenciaException	ModeloMySQL	Mensaje de la operación que falló
UPDATE que no encuentra el cliente	PersistenciaException	ModeloMySQL.actualizarCliente	“No existe un cliente con id …”
Nombre o contacto vacío, precio negativo	IllegalArgumentException	Constructores de las entidades	“Datos inválidos” (la base no se toca)
Venta sin cliente, vendedor o producto	GarantiaException	Constructor de Venta	“No se pudo registrar la venta”

Un ejemplo de cómo se trata en la pantalla de alta de cliente, donde se distinguen los datos inválidos del usuario de los errores de la base:

```
try {
    Cliente nuevo = controlador.agregarCliente(txtNombre.getText(), txtContacto.getText());
    // ... mensaje de confirmación y cierre de la pantalla
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(), "Datos inválidos",
            JOptionPane.WARNING_MESSAGE);
} catch (PersistenciaException ex) {
    Mensajes.errorBaseDeDatos(this, ex);   // mensaje claro + detalle técnico en consola
}
```

Fuente: elaboración propia (VentanaAltaCliente, extracto).

Clase abstracta e interfaz

Clase abstracta Producto (existente)

Producto sigue siendo la abstracción central del dominio: representa “un producto del local” sin saber de qué categoría es, y cada subclase define su período de garantía y su categoría. En el TP4 cumple además una función en la persistencia: se le agregó el método abstracto `getIdCategoria()`, que cada subclase implementa con el id que tiene su categoría en la tabla `categorias` (1, 2 o 3). Así, al guardar un producto en MySQL no hace falta preguntar de qué clase es con `instanceof`: se aprovecha el polimorfismo.

```
// En ModeloMySQL.insertarProducto:
ps.setInt(4, producto.getIdCategoria());   // cada subclase responde su propio id


// En LineaBlanca:
@Override public int getIdCategoria() { return 1; }
```

Fuente: elaboración propia (extractos).

La operación inversa también existe: al leer de MySQL, `ModeloMySQL` crea la subclase que corresponde a cada `id_categoria`.

Interfaz RepositorioPuntoTecno (nueva)

La interfaz `RepositorioPuntoTecno` define qué operaciones de persistencia necesita el sistema (listar, insertar, actualizar, sincronizar), sin decir cómo se realizan. `ModeloMySQL` es su implementación con JDBC. El controlador solo conoce la interfaz:

```
public interface RepositorioPuntoTecno {
    List<Cliente> listarClientes() throws PersistenciaException;
    void insertarCliente(Cliente cliente) throws PersistenciaException;
    void actualizarCliente(Cliente cliente) throws PersistenciaException;
    void insertarVenta(Venta venta) throws PersistenciaException;
    int sincronizarEstadosGarantias() throws PersistenciaException;
    // ... (resto de las operaciones)
}
```

Fuente: elaboración propia (RepositorioPuntoTecno.java, extracto).

Es una interfaz pertinente, y no un requisito formal, porque separa el controlador del motor de base de datos: el controlador y las pantallas no cambiarían si los datos se obtuvieran de otra fuente. La diferencia con la clase abstracta es la siguiente: Producto comparte atributos y comportamiento entre sus subclases (es una relación "es un"), mientras que `RepositorioPuntoTecno` solo declara un contrato que cualquier implementación debe cumplir.

Uso complementario de arreglos y ArrayList

Los **ArrayList** son la estructura principal del sistema porque la cantidad de datos varía: `PuntoTecno` guarda en ArrayList los clientes, vendedores, productos y ventas, y cada consulta a MySQL devuelve su resultado como `List/ArrayList`. El **arreglo** se usa donde el tamaño es fijo y conocido: el sistema tiene exactamente tres categorías de producto.

Ambos se complementan en una operación concreta del arranque, `ControladorPuntoTecno.verificarCategorias()`, que controla que los períodos de garantía de la tabla `categorias` de MySQL coincidan con los de las subclases de Producto. La base entrega sus períodos en un `int[3]` (posición i = categoría con id i + 1), el programa arma otro arreglo con los suyos, y las diferencias, que pueden ser cero, una, dos o tres, se acumulan en un `ArrayList<String>` que se muestra al usuario como advertencia.

```
int[] periodosEnBD = repositorio.periodosGarantiaPorCategoria();      // fijo: 3
int[] periodosEnCodigo = { LineaBlanca.PERIODO_DIAS,
        PequenoElectrodomestico.PERIODO_DIAS, TecnologiaHogar.PERIODO_DIAS };
String[] nombres = { "Línea blanca", "Pequeños electrodomésticos", "Tecnología del hogar" };

List<String> advertencias = new ArrayList<>();                        // variable
for (int i = 0; i < periodosEnCodigo.length; i++) {
    if (periodosEnBD[i] != periodosEnCodigo[i]) {
        advertencias.add(nombres[i] + ": la base de datos indica " + periodosEnBD[i]
                + " días y el programa usa " + periodosEnCodigo[i] + " días ...");
    }
}
```

Fuente: elaboración propia (ControladorPuntoTecno.verificarCategorias, extracto).

No se duplica información: el arreglo solo transporta un dato de la base para compararlo, y la lista guarda únicamente el resultado de la comparación. Este control resuelve un punto real: el período de garantía existe en dos lugares (la tabla `categorias` y las subclases de `Producto`), y el programa avisa si dejan de coincidir. Si difieren, se aplica el valor del programa, que es el que usa Venta para calcular cada garantía.

Funcionalidades implementadas

Opciones del menú y su relación con MySQL

Opción del menú	Pantalla	Requerimiento	Uso de MySQL
1. Registrar venta	VentanaRegistrarVenta	RFS02, RFS03	Guarda venta y garantía en una transacción

Opción del menú	Pantalla	Requerimiento	Uso de MySQL
2. Dar de alta producto	VentanaAltaProducto	RFS04	INSERT en productos
3. Dar de alta cliente	VentanaAltaCliente	RFS05	INSERT en clientes
4. Modificar cliente	VentanaModificarCliente	RFS10	UPDATE en clientes
5. Consultar estado de garantía	VentanaConsultaGarantia	RFS06, RFS07, RFS08	Usa los datos leídos de MySQL
6. Listar ventas	VentanaListarVentas	RFS09	Usa los datos leídos de MySQL
7. Salir	MenuPrincipal	—	—

Las demás decisiones del TP3 se mantienen: los datos se muestran con el estado de garantía calculado, la búsqueda binaria por número de venta y la ordenación por inserción por fecha siguen operando sobre la lista de ventas cargada desde MySQL. El ingreso con usuario y contraseña (RFS01) continúa fuera del alcance de esta etapa.

Pruebas y evidencia de funcionamiento

Cómo compilar y ejecutar

Repositorio del proyecto: https://github.com/Thiago-BF/SeminarioDePracticasDeInformatica-Siglo21/tree/tp4/PuntoTecno

- Iniciar MySQL (por ejemplo desde XAMPP) e importar `sql/puntotecno_tp4.sql`.
- Copiar el `.jar` de MySQL Connector/J en la carpeta `lib`.
- Copiar `config/conexion.properties.example` como `config/conexion.properties` y completar usuario y contraseña.
- Ejecutar ejecutar.bat, o abrir el proyecto en IntelliJ IDEA, marcar `src` como Sources Root, agregar `lib` como biblioteca y ejecutar `PuntoTecnoApp`.

También puede compilarse y ejecutarse desde la consola, ubicándose en la carpeta del proyecto:

```
javac -encoding UTF-8 -cp "lib/*" -d bin src/puntotecno/*.java src/puntotecno/persistencia/*.java
src/puntotecno/controlador/*.java
java -cp "bin;lib/*" puntotecno.PuntoTecnoApp
```

Plan de pruebas

Las pruebas siguientes se ejecutan con la base recién importada y se verifican de dos maneras: en la interfaz y con las consultas de `sql/consultas_demostracion.sql` en MySQL Workbench. Cada prueba tiene su captura de evidencia.

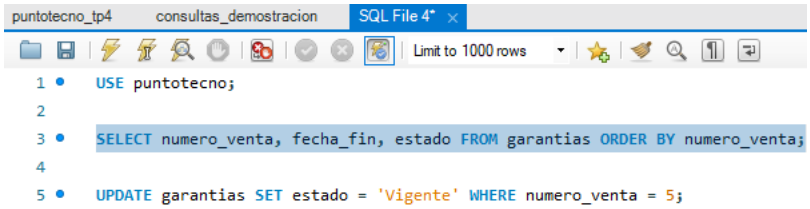
Pruebas de integración con MySQL

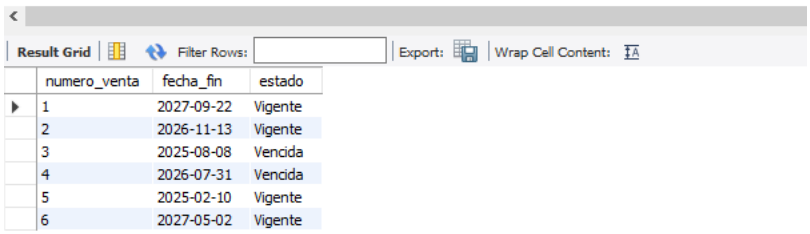
N°	Prueba	Procedimiento	Resultado esperado
PI01	Conexión y lectura	Ejecutar el programa con MySQL iniciado	Se abre el menú y la consola informa “Datos cargados desde MySQL”
PI02	Consulta y presentación	Abrir 6. Listar ventas	Aparecen las 5 ventas de la base; 1 y 2 vigentes, 3, 4 y 5 vencidas
PI03	INSERT de cliente	Dar de alta un cliente y consultar clientes en Workbench	El cliente figura en la tabla con el id 5

N°	Prueba	Procedimiento	Resultado esperado
PI04	UPDATE de cliente	Modificar el contacto de un cliente y consultar en Workbench	El cambio figura en la tabla clientes
PI05	INSERT de producto	Dar de alta un pequeño electrodoméstico y consultar en Workbench	Se guarda con id_categoria = 2
PI06	Venta con garantía	Registrar una venta y consultar ventas y garantias en Workbench	Hay una fila nueva en cada tabla; fecha_fin = fecha de venta + 180 días para un pequeño electrodoméstico
PI07	Persistencia	Cerrar el programa, volver a abrirlo y listar ventas	Las altas y la venta realizadas siguen presentes
PI08	Error de conexión	Detener MySQL y ejecutar el programa	Mensaje “Sin conexión con MySQL”; el programa no inicia y no se cierra con un error sin tratar
PI09	Datos inválidos	Dar de alta un cliente con el nombre vacío	Mensaje “Datos inválidos”; no se agrega ninguna fila a la tabla
PI10	Sincronización de estados	Ejecutar en Workbench UPDATE `garantias SET estado = 'Vigente' WHERE numero_venta = 5;` y abrir el programa	La consola informa 1 garantía corregida y la tabla muestra “Vencida”
PI11	Verificación de categorías	Cambiar periodo_dias de una categoría en Workbench y abrir el programa	Aparece una advertencia que indica la diferencia

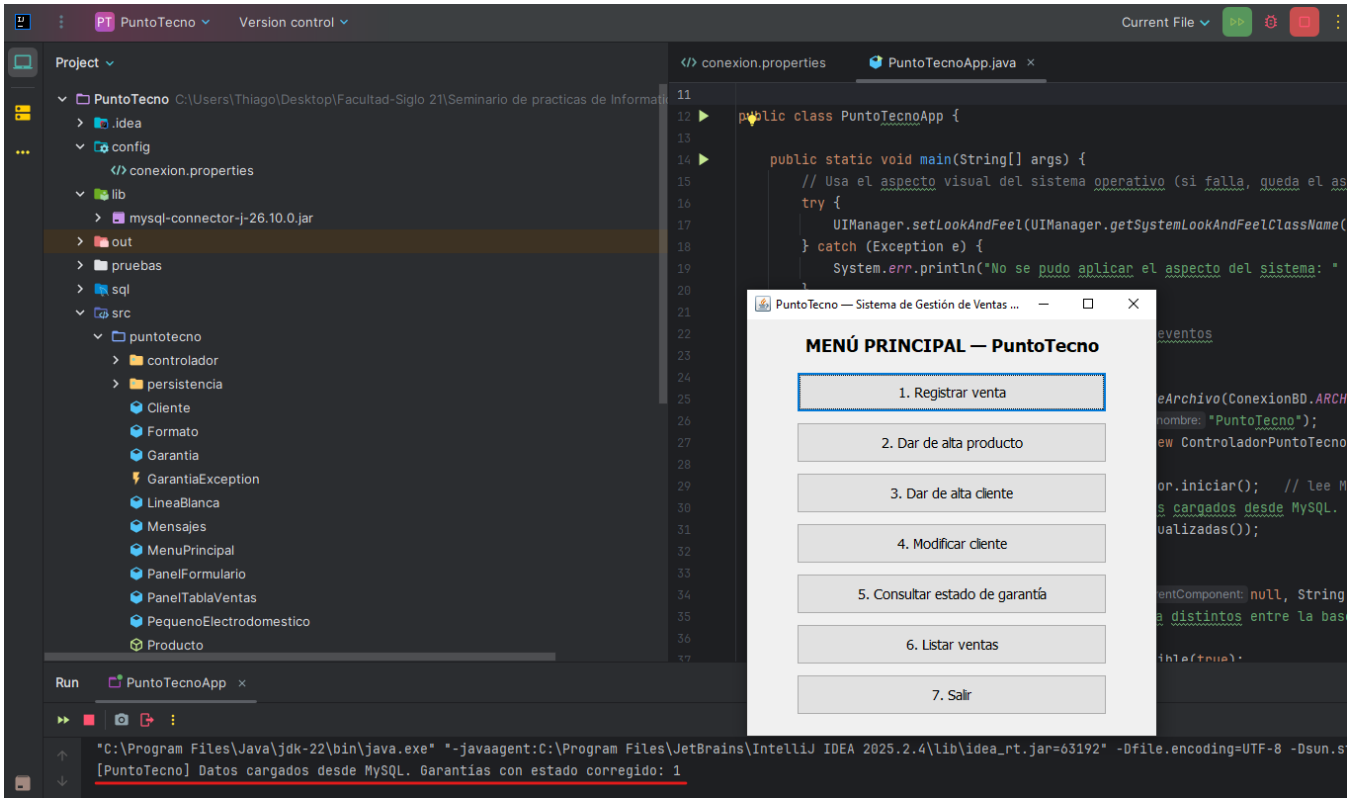
Evidencia de funcionamiento

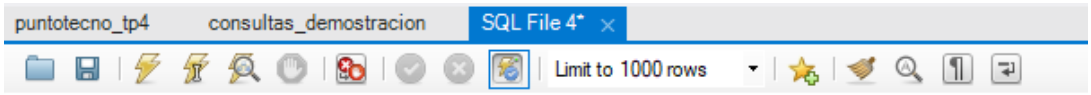
Las siguientes capturas corresponden a las pruebas realizadas el 3/11/2026 sobre la base `puntotecno`, importada desde `sql/puntotecno_tp4.sql`, con el programa conectado a MySQL mediante JDBC. En cada prueba se muestra el resultado en la interfaz y, cuando corresponde, se verifica con una consulta directa en MySQL Workbench. El estado de cada garantía (Vigente o Vencida) se calcula con la fecha del sistema del momento de la ejecución. Capturas:





numero_venta	fecha_fin	estado
1	2027-09-22	Vigente
2	2026-11-13	Vigente
3	2025-08-08	Vencida
4	2026-07-31	Vencida
5	2025-02-10	Vigente
6	2027-05-02	Vigente





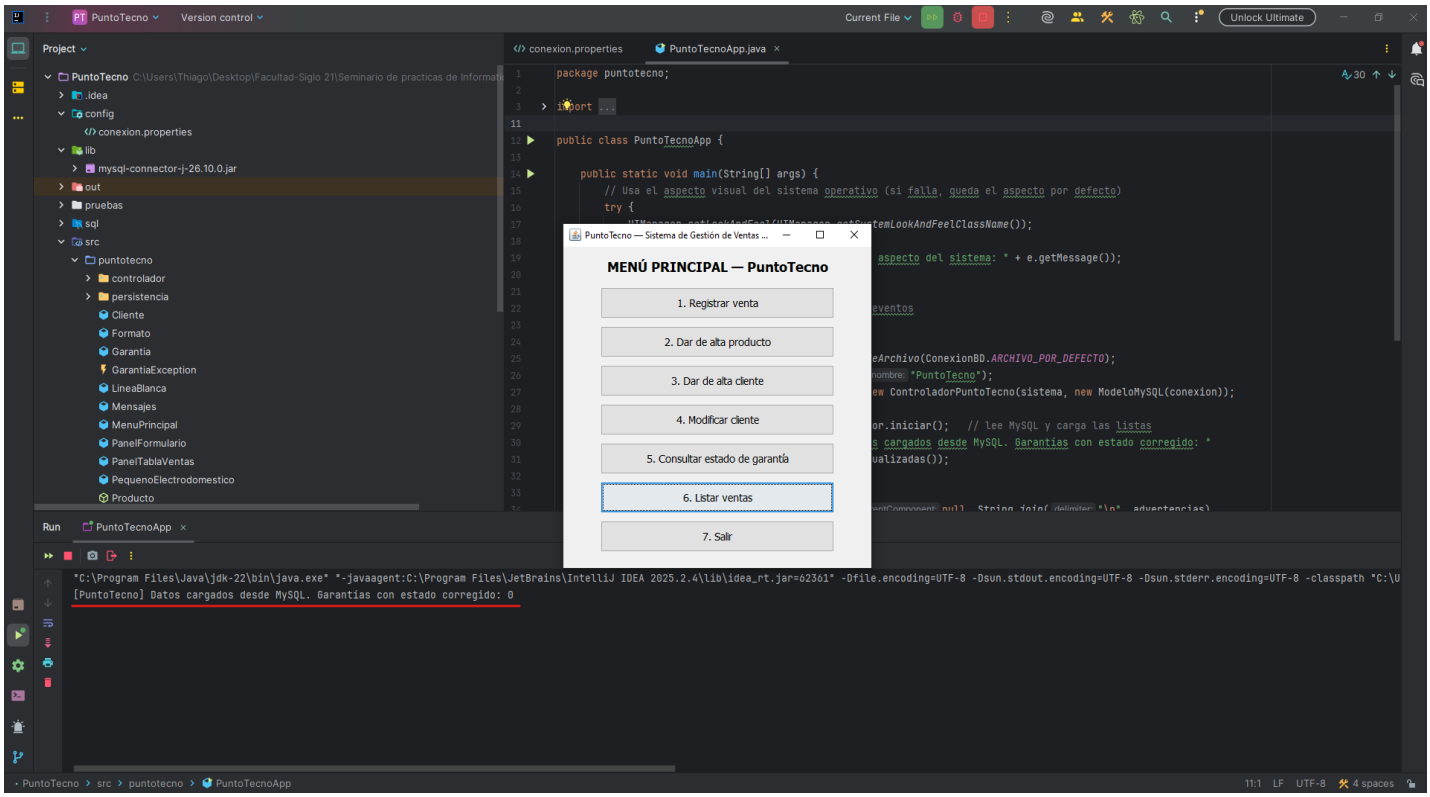

```sql
USE puntotecno;

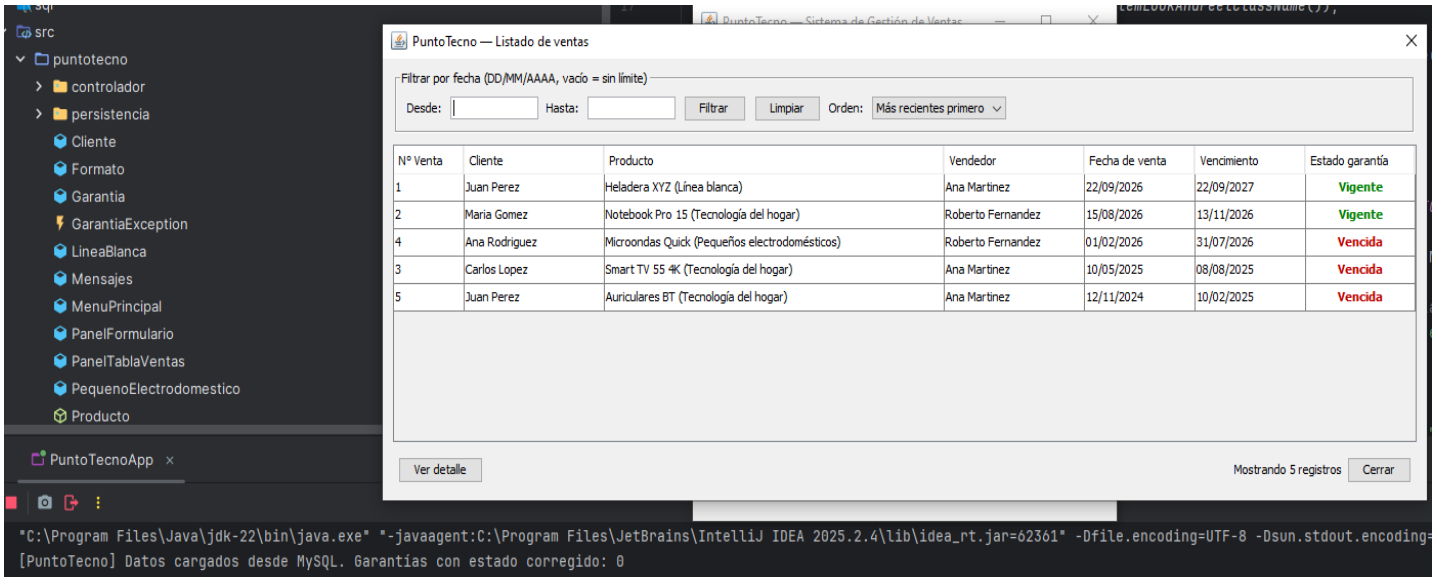
SELECT numero_venta, fecha_fin, estado FROM garantias ORDER BY numero_venta;

UPDATE garantias SET estado = 'Vigente' WHERE numero_venta = 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

numero_venta	fecha_fin	estado
1	2027-09-22	Vigente
2	2026-11-13	Vigente
3	2025-08-08	Vencida
4	2026-07-31	Vencida
5	2025-02-10	Vencida
6	2027-05-02	Vigente


PuntoTecno — Sistema de Gestión de Ventas ...
MENÚ PRINCIPAL — PuntoTecno
1. Registrar venta
2. Dar de alta producto
3. Dar de alta cliente
4. Modificar cliente
5. Consultar estado de garantía
6. Listar ventas
7. Salir
[PuntoTecno] Datos cargados desde MySQL. Garantías con estado corregido: 0


PuntoTecno — Listado de ventas
Filtrar por fecha (DD/MM/AAAA, vacío = sin límite)
Desde:
Hasta:
Filtrar
Limpiar
Orden:
Más recientes primero
N° Venta | Cliente | Producto | Vendedor | Fecha de venta | Vencimiento | Estado garantía
1 | Juan Perez | Heladera XYZ (Línea blanca) | Ana Martinez | 22/09/2026 | 22/09/2027 | Vigente
2 | Maria Gomez | Notebook Pro 15 (Tecnología del hogar) | Roberto Fernandez | 15/08/2026 | 13/11/2026 | Vigente
4 | Ana Rodriguez | Microondas Quick (Pequeños electrodomésticos) | Roberto Fernandez | 01/02/2026 | 31/07/2026 | Vencida
3 | Carlos Lopez | Smart TV 55 4K (Tecnología del hogar) | Ana Martinez | 10/05/2025 | 08/08/2025 | Vencida
5 | Juan Perez | Auriculares BT (Tecnología del hogar) | Ana Martinez | 12/11/2024 | 10/02/2025 | Vencida
Ver detalle
Mostrando 5 registros
Cerrar
[PuntoTecno] Datos cargados desde MySQL. Garantías con estado corregido: 0

MySQL Workbench

Local Instance thiagoBF

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- empresa
- forohub
- jugos_ventas
- **puntotecno**
  - Tables
    - categorias
    - clientes
    - garantias
    - productos
    - vendedores
    - ventas
  - Views
  - Stored Procedures
  - Functions
- sys
- vollmed_api

Administration Schemas

Information

No object selected

puntotecno_tp4 consultas_demostracion

Limit to 1000 rows

```
13    JOIN productos p   ON v.id_producto = p.id_producto
14    JOIN garantias g   ON v.numero_venta = g.numero_venta
15    ORDER BY v.numero_venta;
16
17    -- 2) Clientes (verifica "Dar de alta cliente" y "Modificar cliente")
18    SELECT * FROM clientes ORDER BY id_cliente;
19
20    -- 3) Productos con su categoria (verifica "Dar de alta producto")
21    SELECT p.id_producto, p.nombre, p.precio, c.nombre AS categoria, c.periodo_dias
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_cliente	nombre	contacto
1	Juan Perez	3564-111222
2	Maria Gomez	3564-222333
3	Carlos Lopez	3564-333444
4	Ana Rodriguez	3564-444555
5	Thiago Beber Feil	3446526921
NULL	NULL	NULL

clientes 1

Output

Action Output

#	Time	Action	Message
1	11:08:09	SELECT * FROM clientes ORDER BY id_cliente LIMIT 0, 1000	5 row(s) returned

PuntoTecno — Sistema de Gestión de Ventas ...

MENÚ PRINCIPAL — PuntoTecno

1. Registrar venta

PuntoTecno — Modificar cliente

Cliente: Thiago Beber (3446526941)

Nombre: Thiago Beber

Contacto (teléfono): 3446526941

Guardar cambios Cerrar

5. Consultar estado de garantía

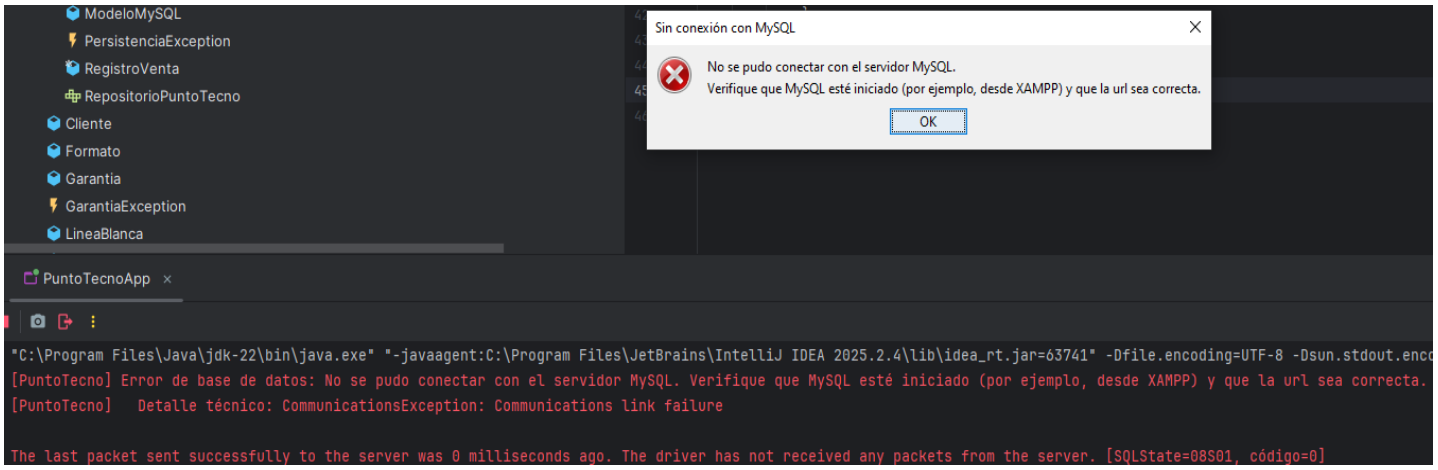
Modificar cliente

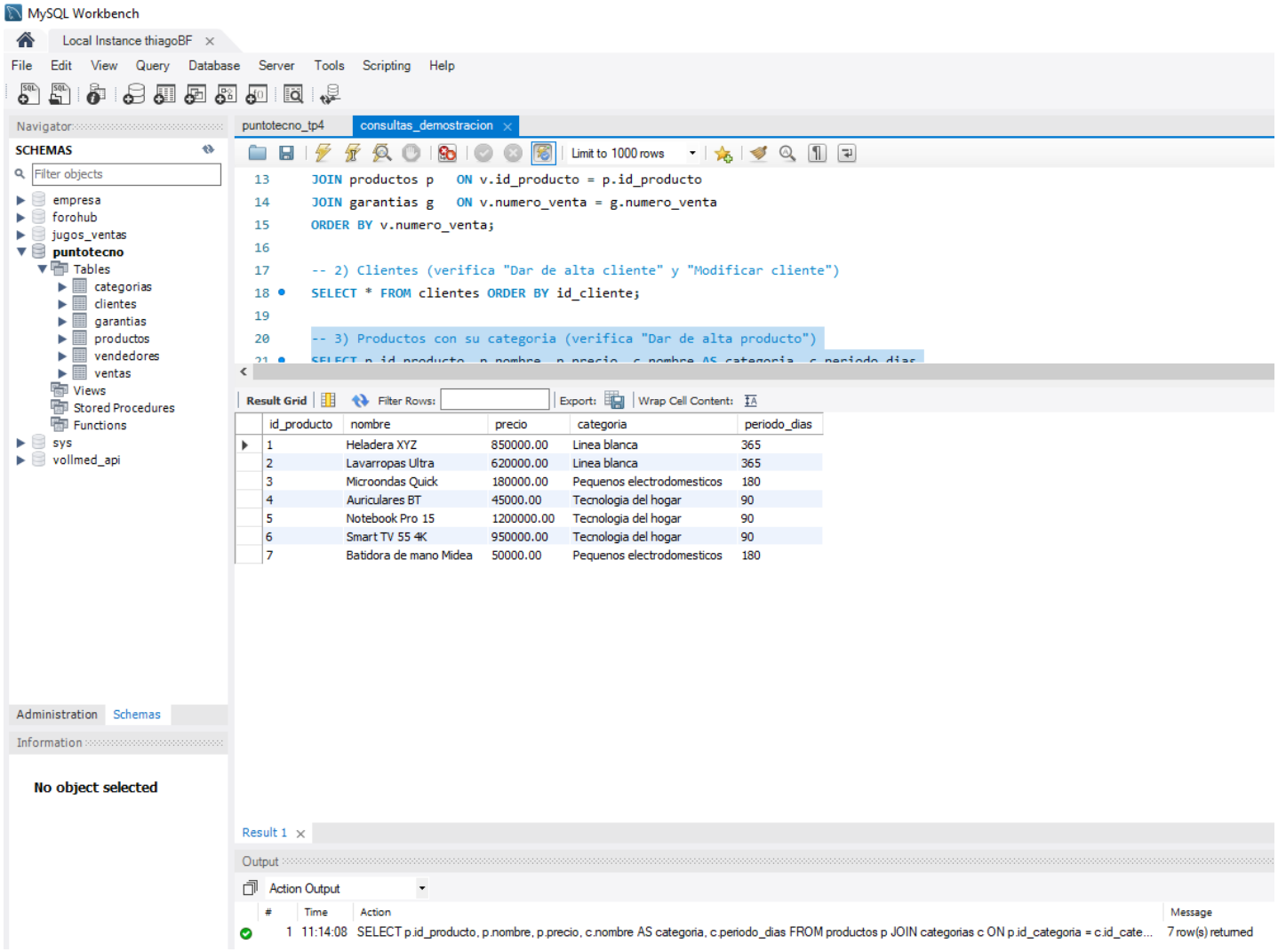
Datos del cliente actualizados.

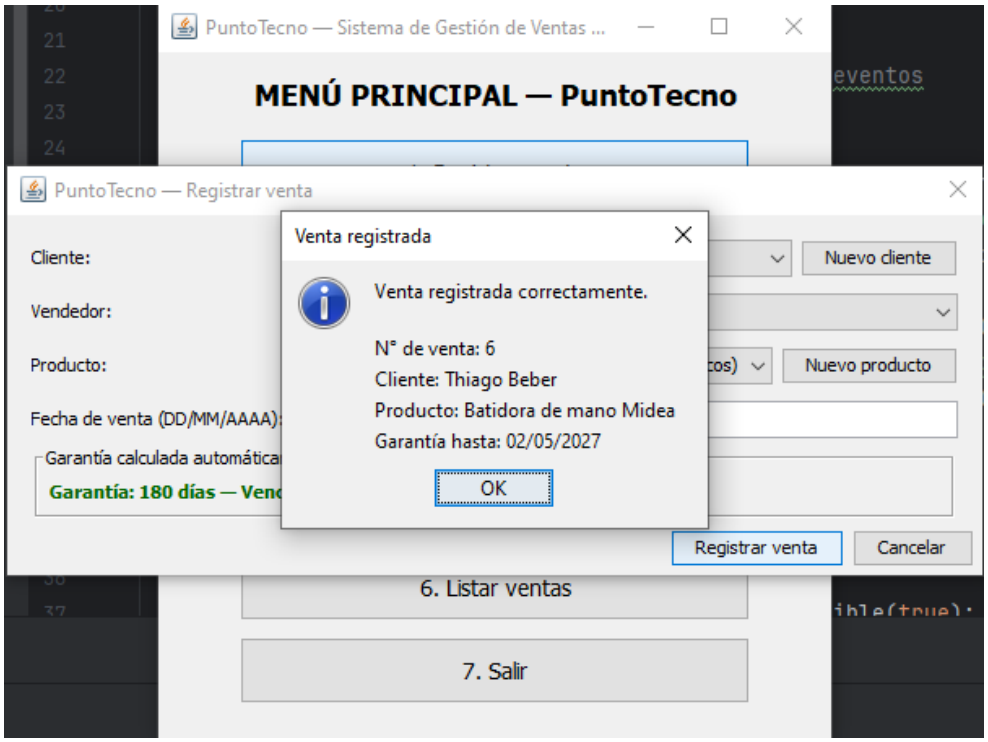
OK

Result Grid | Filter Rows:

id_cliente	nombre	contacto
1	Juan Perez	3564-111222
2	Maria Gomez	3564-222333
3	Carlos Lopez	3564-333444
4	Ana Rodriguez	3564-444555
5	Thiago Beber	3446526941
NULL	NULL	NULL


ModeloMySQL
PersistenciaException
RegistroVenta
RepositorioPuntoTecno
Cliente
Formato
Garantia
GarantiaException
LineaBlanca
Sin conexión con MySQL
No se pudo conectar con el servidor MySQL.
Verifique que MySQL esté iniciado (por ejemplo, desde XAMPP) y que la url sea correcta.
OK
PuntoTecnoApp
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.4\lib\idea_rt.jar=63741" -Dfile.encoding=UTF-8 -Dsun.stdout.enco
[PuntoTecno] Error de base de datos: No se pudo conectar con el servidor MySQL. Verifique que MySQL esté iniciado (por ejemplo, desde XAMPP) y que la url sea correcta.
[PuntoTecno] Detalle técnico: CommunicationsException: Communications link failure
The last packet sent successfully to the server was 0 milliseconds ago. The driver has not received any packets from the server. [SQLState=08S01, código=0]


MySQL Workbench
Local Instance thiagoBF
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
empresa
forohub
jugos_ventas
puntotecno
Tables
categorias
clientes
garantias
productos
vendedores
ventas
Views
Stored Procedures
Functions
sys
vollmed_api
Administration Schemas
Information
No object selected
puntotecno_tp4
consultas_demostracion
Limit to 1000 rows
13 JOIN productos p ON v.id_producto = p.id_producto
14 JOIN garantias g ON v.numero_venta = g.numero_venta
15 ORDER BY v.numero_venta;
16
17 -- 2) Clientes (verifica "Dar de alta cliente" y "Modificar cliente")
18 SELECT * FROM clientes ORDER BY id_cliente;
19
20 -- 3) Productos con su categoria (verifica "Dar de alta producto")
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
id_producto nombre precio categoria periodo_dias
1 Heladera XYZ 850000.00 Linea blanca 365
2 Lavarropas Ultra 620000.00 Linea blanca 365
3 Microondas Quick 180000.00 Pequenos electrodomesticos 180
4 Auriculares BT 45000.00 Tecnologia del hogar 90
5 Notebook Pro 15 1200000.00 Tecnologia del hogar 90
6 Smart TV 55 4K 950000.00 Tecnologia del hogar 90
7 Batidora de mano Midea 50000.00 Pequenos electrodomesticos 180
Result 1
Output
Action Output
Time Action Message
1 11:14:08 SELECT p.id_producto, p.nombre, p.precio, c.nombre AS categoria, c.periodo_dias FROM productos p JOIN categorias c ON p.id_categoria = c.id_cate... 7 row(s) returned



PuntoTecno — Listado de ventas

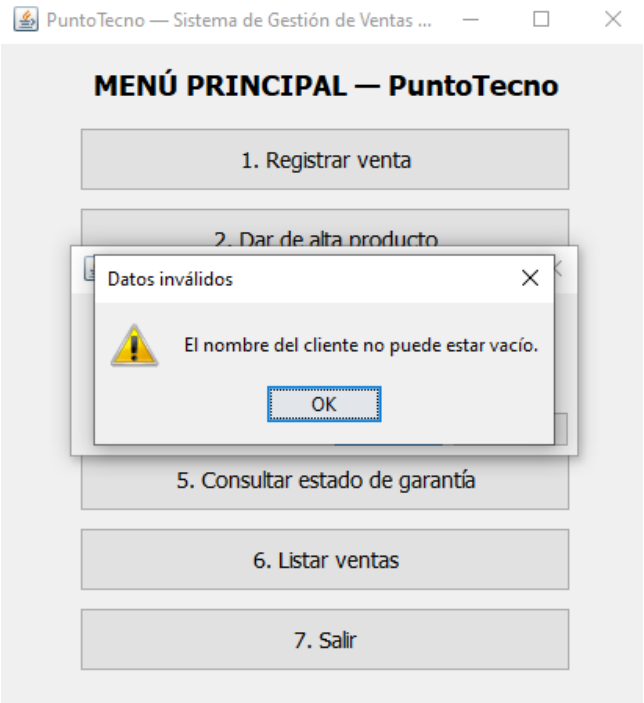
Filtrar por fecha (DD/MM/AAAA, vacío = sin límite)

Desde: Hasta: Filtrar Limpiar Orden: Más recientes primero

N° Venta	Cliente	Producto	Vendedor	Fecha de venta	Vencimiento	Estado garantía
6	Thiago Beber	Batidora de mano Midea (Pequeños electrodomésticos)	Roberto Fernandez	03/11/2026	02/05/2027	Vigente
1	Juan Perez	Heladera XYZ (Línea blanca)	Ana Martinez	22/09/2026	22/09/2027	Vigente
2	Maria Gomez	Notebook Pro 15 (Tecnología del hogar)	Roberto Fernandez	15/08/2026	13/11/2026	Vigente
4	Ana Rodriguez	Microondas Quick (Pequeños electrodomésticos)	Roberto Fernandez	01/02/2026	31/07/2026	Vencida
3	Carlos Lopez	Smart TV 55 4K (Tecnología del hogar)	Ana Martinez	10/05/2025	08/08/2025	Vencida
5	Juan Perez	Auriculares BT (Tecnología del hogar)	Ana Martinez	12/11/2024	10/02/2025	Vencida

Ver detalle

Mostrando 6 registros Cerrar

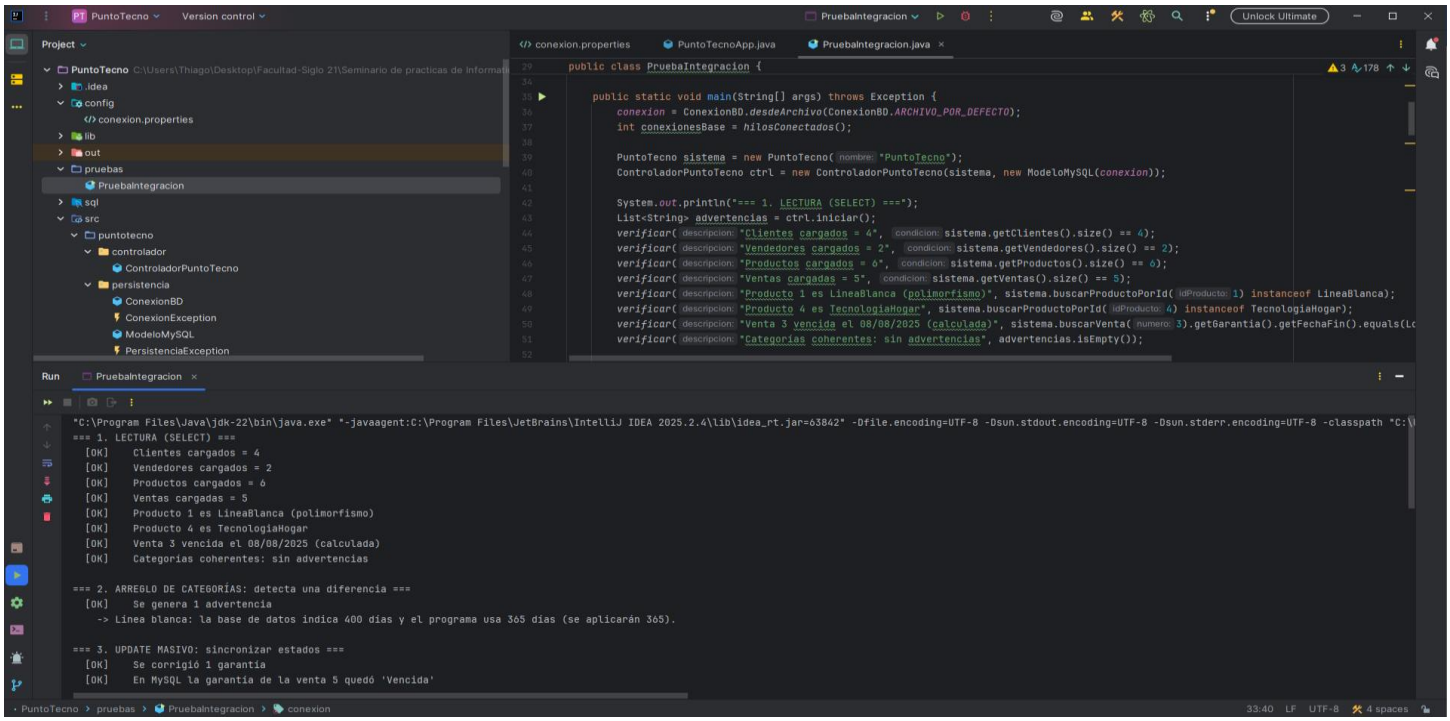


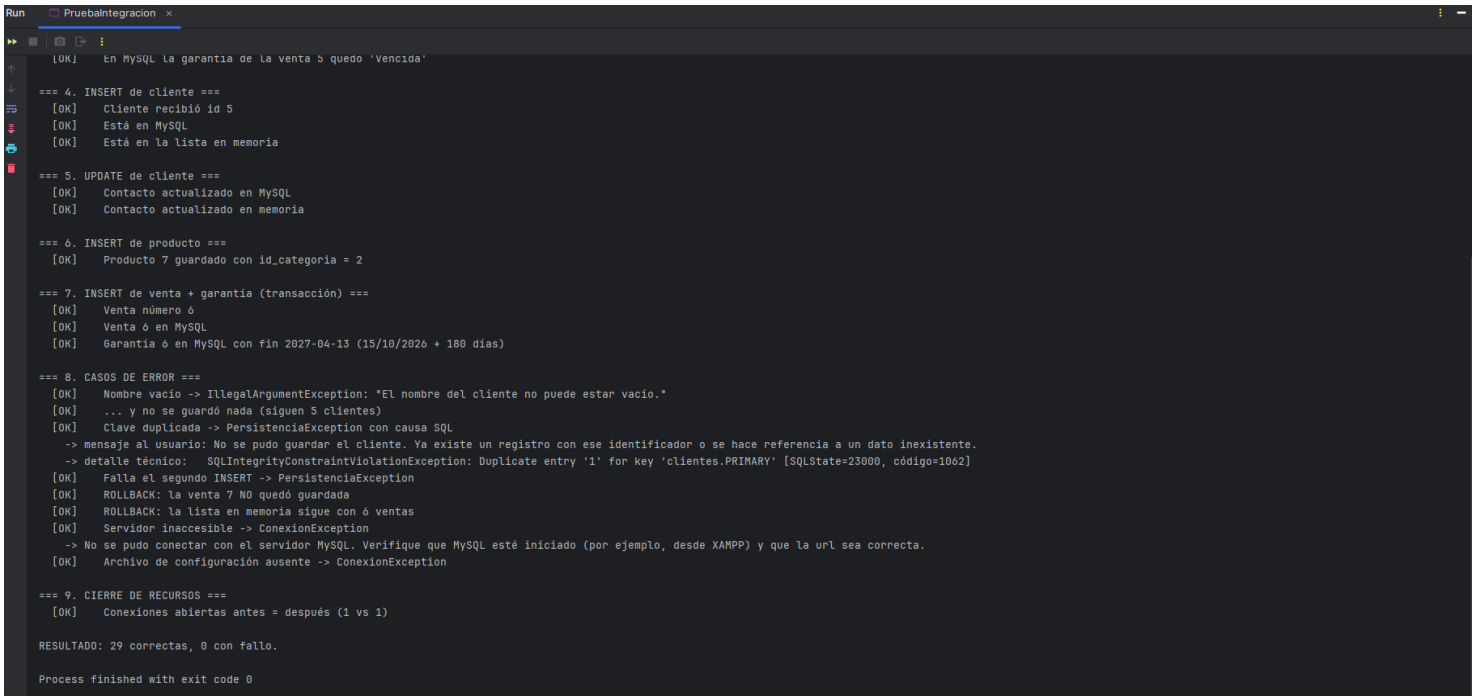
Prueba de integración por consola

Además de las pruebas desde la interfaz, el proyecto incluye pruebas/PruebaIntegracion.java, que ejecuta sin interfaz gráfica una batería de 29 verificaciones directamente contra la base: lectura de datos, INSERT y UPDATE con comprobación en MySQL, la transacción con `rollback` (se fuerza una falla en el segundo INSERT y se comprueba que la venta no quedó guardada), clave duplicada, servidor inaccesible, archivo de configuración ausente y cierre de conexiones (la cantidad de conexiones abiertas en el servidor es la misma antes y después de repetir la carga 30 veces). Se ejecuta con la base recién importada:

```
javac -encoding UTF-8 -cp "lib/*" -d bin src/puntotecno/*.java src/puntotecno/persistencia/*.java
src/puntotecno/controlador/*.java pruebas/PruebaIntegracion.java
java -cp "bin;lib/*" PruebaIntegracion
```

Resultado esperado al final de la salida: “RESULTADO: 29 correctas, 0 con fallo.”





Fuente: elaboración propia.

Limitaciones y trabajo futuro

- Los datos se cargan una vez al iniciar el programa. Si en el futuro varias terminales trabajaran a la vez (RNF05), cada una vería los cambios de las otras recién al reiniciarse. Los identificadores se calculan contra MySQL y una colisión se detecta como error sin pisar datos.
- El ingreso con usuario y contraseña (RFS01) queda para una etapa posterior.
- El sistema abre una conexión por operación. Para un volumen mayor podría incorporarse un pool de conexiones.

Conclusión

El desarrollo de PuntoTecno permitió pasar de un problema real, el registro disperso de ventas y el cálculo manual de garantías en planillas, a un sistema que centraliza esa información en una base de datos MySQL y permite saber en el momento si la garantía de un producto sigue vigente. Para quienes trabajamos en el local, entre ellos yo, la aplicación puede ser una gran ayuda: cada venta se registra una sola vez, la fecha de vencimiento se calcula sola según la categoría del producto y ya no hace falta buscar en planillas antiguas ni hacer cuentas a mano cada vez que alguien consulta. Esto reduce los errores y acorta los tiempos de atención.

También puede beneficiar a los clientes, que reciben una respuesta inmediata y confiable sobre el estado de su garantía y ya no tendrían que llamar o volver al local para preguntar si su producto sigue en vigencia. Desde lo técnico, el uso del patrón MVC, las excepciones propias para los errores de la base de datos, la interfaz y la clase abstracta, y el uso complementario de arreglos y ArrayList dejan un sistema ordenado y preparado para crecer. Entre los próximos pasos quedan el ingreso con usuario y contraseña (RFS01) y la actualización de los datos cuando trabajen varias terminales a la vez.